



**UNIVERSIDADE FEDERAL DE
MINAS GERAIS**

TRABALHO FINAL DE AUTOMAÇÃO EM TEMPO REAL

Entrega 2: Relatório Final

Ana Clara Melado, 2023421386

Bruno Araujo Pinto, 2023038965

Mariana Villefort Rabelo, 2022061033

Belo Horizonte
22 de junho de 2026

1 Introdução

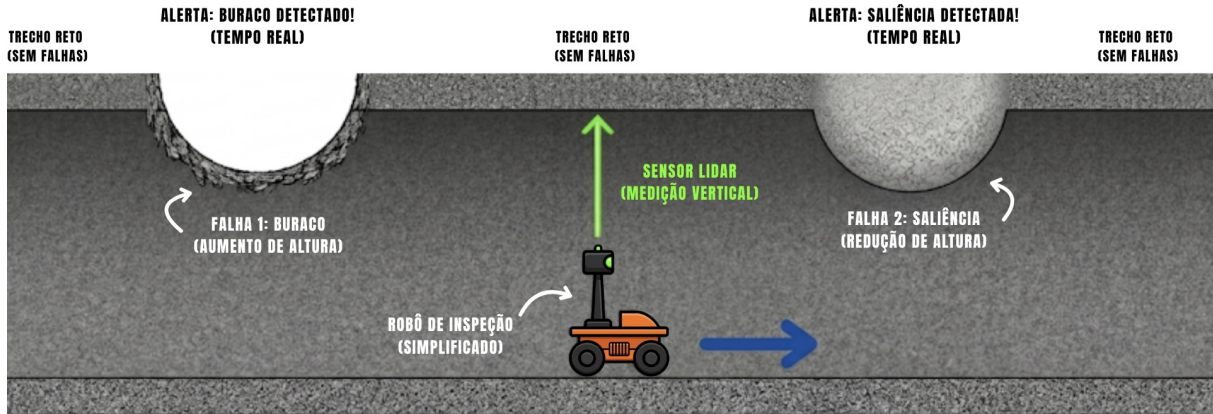


Figura 1: Túnel (vista lateral 2D)

A inspeção periódica de túneis e estruturas civis é fundamental para garantir a segurança operacional e estrutural desses ambientes. Entretanto, métodos tradicionais de inspeção manual apresentam limitações relacionadas ao custo, ao tempo de execução e à exposição de operadores a ambientes potencialmente perigosos. Nesse contexto, sistemas autônomos de inspeção vêm sendo cada vez mais utilizados, permitindo a coleta contínua de dados e a identificação precoce de falhas estruturais.

Este trabalho tem como objetivo o desenvolvimento da arquitetura de um sistema embarcado para inspeção automática de túneis, utilizando conceitos de Automação em Tempo Real, programação concorrente e sincronização entre tarefas. O sistema proposto simula o funcionamento de um robô autônomo responsável por percorrer um túnel realizando medições da superfície do teto por meio de sensores simulados, como encoder e LIDAR, possibilitando a detecção de irregularidades estruturais.

A implementação desenvolvida na primeira etapa do projeto concentra-se principalmente nas tarefas responsáveis pelo controle de navegação, cálculo de distância percorrida, reconstrução da superfície do túnel e coleta de dados. Para isso, foram utilizados processos e múltiplas threads em C++, além de mecanismos de sincronização como `mutex` e `condition_variable`, garantindo acesso seguro aos buffers compartilhados e evitando condições de corrida durante a troca de informações entre produtor e consumidor.

Além da implementação dos buffers compartilhados, o projeto também realiza testes preliminares de sincronização e bloqueio, verificando situações de buffer vazio e buffer cheio. Esses testes permitem validar o comportamento concorrente do sistema e demonstrar a correta coordenação entre as tarefas executadas em paralelo, aspecto essencial em aplicações de tempo real.

Por fim, esta etapa do trabalho estabelece a base arquitetural necessária para as próximas implementações do sistema completo, incluindo comunicação remota, interface gráfica de operação e integração com protocolos de comunicação.

2 Proposta de Arquitetura

A proposta de arquitetura apresentada na primeira etapa do trabalho foi totalmente remodelada, pois não se adequava aos requisitos de utilização da biblioteca Boost. Além

disso, identificou-se o acúmulo de tarefas na fila entre as *threads*. Na abordagem inicial, o travamento das tarefas era frequente, dado que o período da tarefa *distancia_percorrida()* é de 20 ms, enquanto o da tarefa *reconstrucao_teto()* é de 100 ms. Consequentemente, a taxa de escrita no **BUFFER_NAVEGACAO** é significativamente superior à sua taxa de leitura. Assim, ao utilizar a instrução *wait()* em *distancia_percorrida()*, o sistema entrava em um *deadlock*. Para evitar esse cenário, quando uma tarefa agendada não está apta para execução, ela é finalizada forçadamente, sendo contabilizada como uma perda de prazo (*miss*).

Ademais, implementou-se o uso de *strands* para otimizar o aproveitamento da biblioteca Boost e evitar condições de corrida no acesso aos *buffers*. A classe `io_service::strand` atua como um serializador de contexto de I/O, gerindo o agendamento de execuções. Por meio da função `boost::post`, é possível enfileirar novas tarefas de forma segura. Isso garante que as tarefas sejam executadas sequencialmente na *strand*, eliminando a preempção concorrente e as condições de corrida sem a necessidade de bloqueios explícitos.

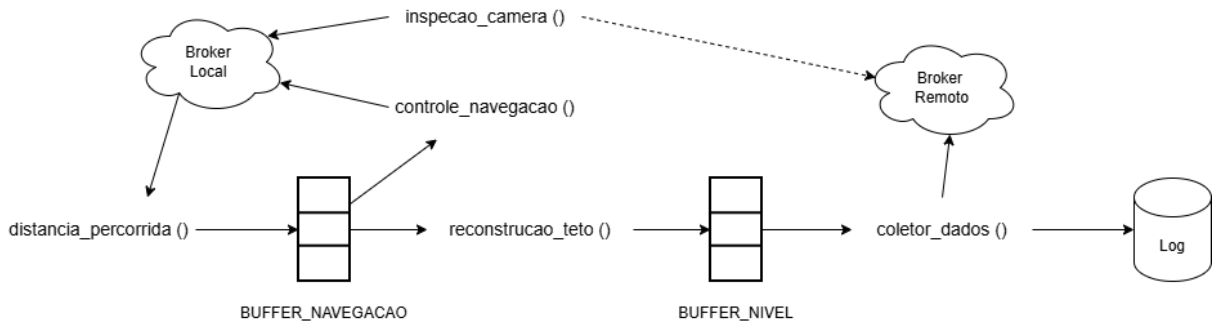


Figura 2: Proposta de arquitetura para o trabalho prático. É possível identificar a relação dos buffers com as funções.

2.1 Periodização

A sincronização das tarefas como introduzido anteriormente foi totalmente readequado para a biblioteca Boost. Foram criadas três strands: *strand_navegacao*, *strand_processamento* e *strand_camera*. As duas primeiras gerenciam as tarefas como ilustrado na figura 3. Já a terceira gerencia apenas a utilização da câmera que não interage diretamente com as outras tarefas.

Ademais, como o período das strands são conflitantes ($T_{distancia_percorrida} = 20ms$ e $T_{reconstrucao_teto} = 100ms$), a tarefa *distancia_percorrida* acontece em único ciclo, enquanto a tarefa *reconstrucao_teto* executa em "lotes", até que se encontre uma falha. Dessa forma, é possível facilitar a sincronização de ambas as filas. A figura 3 demonstra a divisão das strands.

2.2 Comunicação

Conforme exigido pelo escopo do trabalho, o projeto contempla três requisitos principais: o uso de comunicação interprocessos (IPC), o envio de sinais (*signals*) entre duas tarefas para o acionamento da câmera e a integração de um *broker* para a comunicação com as interfaces.

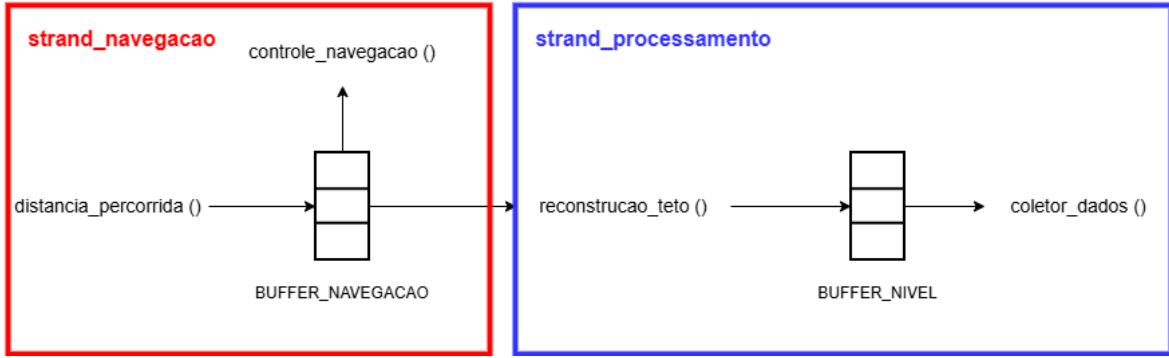


Figura 3: Divisão das strands.

2.2.1 Implementação da Memória Compartilhada

Após a criação do arquivo de memória, a classe `file_mapping` é utilizada para associá-lo ao processo, enquanto a classe `mapped_region` realiza o mapeamento efetivo do arquivo no espaço de endereçamento da aplicação. O endereço retornado por essa região é convertido para um ponteiro da estrutura `MemoriaCompartilhada`, permitindo que os campos sejam acessados diretamente pelas diferentes tarefas.

A estrutura compartilhada reúne variáveis relacionadas aos sensores, atuadores, estados e comandos do robô. Entre elas, encontram-se o estado do encoder, a leitura do sensor LiDAR, o comando de acionamento da câmera, o valor de aceleração, o estado de inspeção, o modo de operação e o *setpoint* de velocidade. Dessa forma, a estrutura representa a interface comum entre a simulação, o controle de navegação, a operação remota e os módulos de comunicação.

A tarefa *comando_navegacao* e as tarefas executadas no processo principal operam em processos distintos. Após a criação do processo por meio da função `fork()`, o processo filho abre novamente o mesmo arquivo mapeado e obtém acesso à estrutura compartilhada. Assim, uma alteração realizada por um processo pode ser observada pelos demais processos que possuem a mesma região de memória mapeada.

Esse mecanismo é utilizado, por exemplo, para que o processo responsável pelo comando de navegação atualize o modo de operação e o *setpoint* de velocidade, enquanto a tarefa de controle consulta esses valores para calcular a atuação do robô. De maneira semelhante, a interface de simulação escreve os valores correspondentes ao encoder e ao LiDAR, enquanto o núcleo em C++ utiliza essas informações no cálculo da distância e na reconstrução do teto.

A interface gráfica desenvolvida em Python também acessa o arquivo de Memória Compartilhada. Para isso, foi criada em Python uma estrutura equivalente à estrutura utilizada no código C++, mantendo a mesma ordem e os mesmos tipos básicos dos campos. O arquivo é aberto em modo de leitura e escrita e mapeado por meio do módulo `mmap`. Em seguida, o módulo `ctypes` permite interpretar os bytes da região mapeada como uma estrutura de dados. A equivalência entre as estruturas declaradas em C++ e Python é necessária para garantir que os dois programas interpretem corretamente as mesmas posições da memória.

Além da interface de simulação, os clientes MQTT também utilizam os valores armazenados na Memória Compartilhada. O cliente `robot_publisher` realiza a leitura periódica dos sensores e estados do robô e publica essas informações no *broker*. Por sua vez, o

cliente `robot_subscriber` recebe os comandos enviados pela interface de operação remota e atualiza os campos correspondentes na Memória Compartilhada.

Como diferentes tarefas podem acessar os `mestextttmutex`, garantindo que o publicador não leia a estrutura durante uma atualização realizada pelo subscritor. Nas demais partes do sistema, a organização das tarefas em processos, *strands* de um `mutex`, garantindo que o publicador não leia a estrutura durante uma atualização realizada pelo subscritor. Nas demais partes do sistema, a organização das tarefas em processos, *strands* e regiões críticas impede a ocorrência de condições de corrida.

2.2.2 Signals

A comunicação por *signals* foi empregada para notificar a detecção de falhas no túnel. Na tarefa *reconstrucao_teto*, caso seja identificada uma variação estrutural acima do esperado, o sinal `SIGUSR1` é emitido, conforme ilustrado na Figura 4. Uma rotina de tratamento (*signal_handler*) intercepta esse sinal e atua imediatamente no sistema, desacelerando o veículo e ativando a câmera de inspeção.

2.2.3 MQTT

Para o *broker*, foi utilizado o Mosquitto, rodando localmente no endereço `tcp://localhost:1883`. A comunicação foi estruturada em dois clientes distintos: `robot_publisher`, responsável por publicar os dados do robô à interface remota, e `robot_subscriber`, responsável por receber comandos enviados pela interface remota.

Os tópicos de publicação (*Robot* $\rightarrow$ *Interface Remota*) são:

- `robot/sensors/lidar` — leitura atual do sensor LiDAR;
- `robot/sensors/encoder` — estado do encoder de distância;
- `robot/status/inspection` — status da inspeção por câmera;
- `robot/status/navigation` — status do modo de navegação;
- `robot/data/tunnel_profile` — perfil acumulado do teto do túnel;
- `robot/data/velocity` — velocidade atual do robô;
- `robot/data/confidence` — nível de confiança da leitura atual.

Os tópicos de subscrição (*Interface Remota* $\rightarrow$ *Robot*) são:

- `robot/commands/navigation` — comandos de movimentação (frente, ré, parar);
- `robot/commands/mode` — alternância entre modo automático e manual;
- `robot/commands/velocity` — ajuste direto do *setpoint* de velocidade;
- `robot/commands/camera` — ligar ou desligar a câmera de inspeção;
- `robot/commands/threshold` — ajuste do limiar de detecção de anomalia.

A qualidade de serviço (QoS) foi definida por criticidade: dados de sensores e comandos utilizam QoS 1 (*at least once*), garantindo entrega sem duplicação excessiva; dados de status utilizam QoS 0 (*at most once*), priorizando baixa latência.

O MQTT atua como ponte entre o núcleo C++ de tempo real e a interface remota em Python. O `robot_publisher` lê periodicamente os dados da Memória Compartilhada e os publica no broker; o `robot_subscriber`, por sua vez, recebe os comandos enviados pela interface remota e os escreve de volta na Memória Compartilhada, onde serão consumidos pelas tarefas de controle na próxima iteração. Dessa forma, o MQTT não interfere diretamente na lógica de tempo real — ele opera como uma camada de transporte assíncrona, desacoplando o ritmo da interface remota do ritmo das tarefas periódicas do sistema.

A frequência de publicação de cada tópico é determinada pela tarefa que produz os dados na Memória Compartilhada. Os tópicos `robot/sensors/lidar`, `robot/sensors/encoder`, `robot/data/velocity` e `robot/data/confidence` são publicados a cada 20 ms, cadência da tarefa `distancia_percorrida`. Já `robot/data/tunnel\_profile`, `robot/status/navigation` e `robot/status/inspection` são atualizados a cada 80 ms, alinhados com o período das tarefas `reconstrucao_teto` e `controle_navegacao`. Essa distinção evita que tópicos de menor variação temporal sobrecarreguem o broker com publicações desnecessárias.

```
float variacao = std::abs(media - baseline);
if (variacao > shm->variacao_severa) {
    if (!shm->e_inspecao) { // dispara apenas uma vez por anomalia
        shm->e_inspecao = true;
        shm->o_liga_camera = true;
        if (shm->e_automatico) // desacelera apenas em modo automático
            shm->o_aceleracao = -30;
        kill(getpid(), SIGUSR1);
        log_message("RECONSTRUCAO", "Variacao severa detectada: " + std::to_string(variacao));
    }
}
```

Figura 4: Trecho de código com a chamada do signal.

2.3 Interfaces Gráficas

A primeira interface gráfica, apresentada na Figura 5, foi estruturada para atuar como a representação do ambiente físico e do veículo de inspeção, servindo como a planta virtual do sistema. A interação contínua com as tarefas de controle é realizada via Memória Compartilhada, utilizando uma estrutura de variáveis que espelha fielmente os dados do controlador. Dessa forma, a aplicação é capaz de receber e aplicar comandos de movimentação, como a aceleração e a velocidade estipuladas, enquanto escreve simultaneamente na memória o estado atual do robô e as medições do ambiente para que o sistema reaja adequadamente.

Internamente, este *script* também é responsável pela simulação da física do movimento e pela geração dos dados sensoriais. Ele cria as anomalias no teto do túnel (como buracos e protuberâncias) e emula o comportamento do *encoder* para medir a distância percorrida, além de um LiDAR, que calcula a distância até o teto adicionando ruídos característicos de sensores reais. Além de renderizar a simulação graficamente, a interface captura entradas do teclado, permitindo que o usuário interaja diretamente com o sistema local para alterar estados, como alternar entre os modos manual e automático ou ajustar a velocidade do veículo.

Em complemento à planta virtual, uma segunda aplicação foi desenvolvida para atuar como um sistema supervisor e de operação remota, conforme ilustrado na Figura 6. Distanciando-se do controle de baixo nível e da simulação física direta, a troca de informações desta interface com o sistema ocorre de forma distribuída por meio do MQTT descrito anteriormente.

Além disso, foi incorporado um painel analítico dinâmico: a partir dos pacotes MQTT, a interface renderiza em tempo real um gráfico do perfil do teto e exibe a leitura do lidar. O operador também pode realizar alterações nos modos por meio do teclado: digitando M para modo Manual, A para modo automático, C desligar ou ligar a câmera e assim por diante.

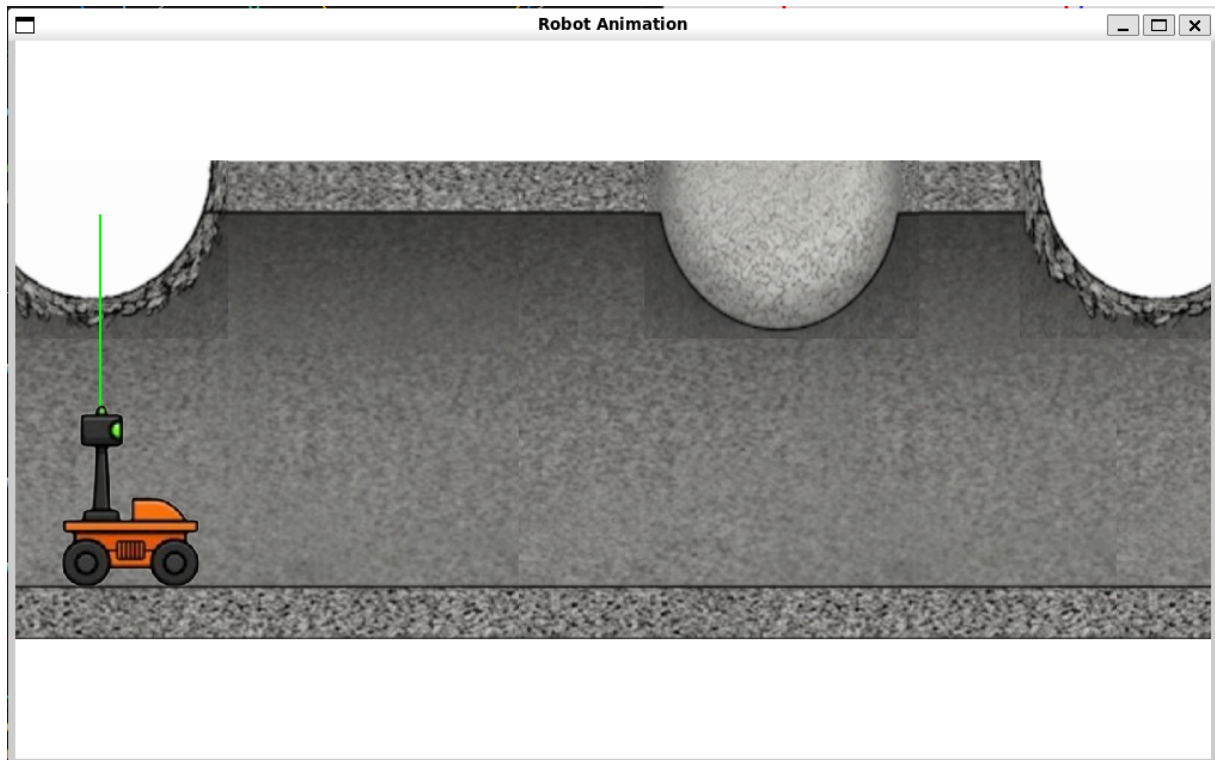


Figura 5: Interface da simulação.

2.4 Dados

O armazenamento dos dados coletados está sendo feita por meio de arquivos, devido também à simplicidade de implementação. A cada ciclo ele coleta os dados armazenados na Memória Compartilhada e salva em csv, como relatado na figura 7.

3 Testes Unitários

Para validar o comportamento isolado de cada módulo do sistema, foram desenvolvidos testes unitários em duas frentes: uma suíte em C++ cobrindo a lógica embarcada do núcleo de tempo real, e uma suíte em Python cobrindo a simulação física e os algoritmos de processamento de dados. Os testes são executados via `make test`, que compila e roda ambas as suítes sequencialmente.

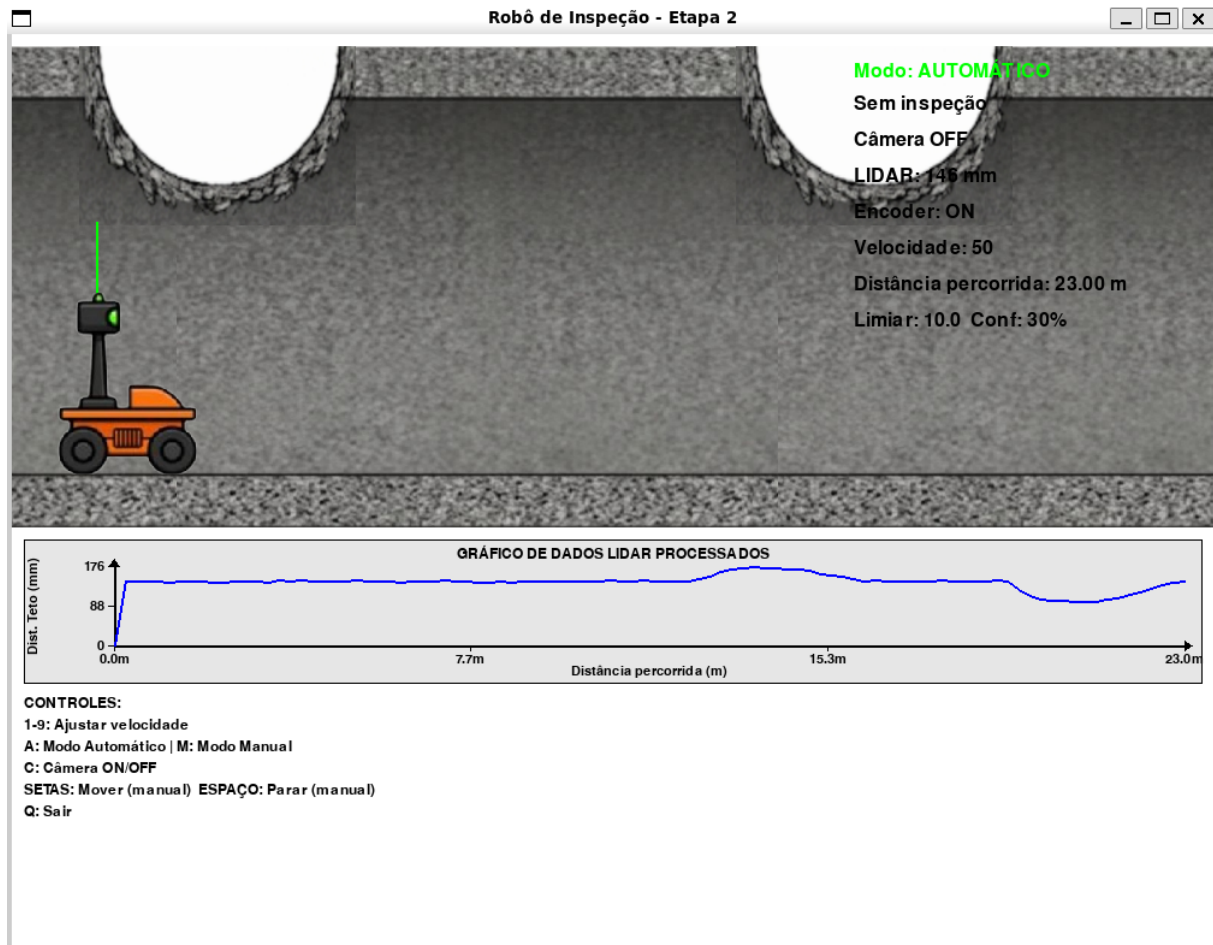


Figura 6: Interface da operação remota.

3.1 Testes em C++

Os testes em C++ foram implementados no arquivo `src/test_sistema.cpp`, utilizando um *framework* minimalista próprio — uma função `check(condição, nome)` que contabiliza aprovações e falhas e retorna código de saída 1 caso algum teste falhe. Foram implementados 35 testes distribuídos em 7 grupos:

- **Encoder:** verifica a detecção de transições de borda ($0 \rightarrow 1$ e $1 \rightarrow 0$), a ausência de incremento em estado repetido e o acúmulo correto de 10 transições consecutivas;
- **Filtro de média móvel:** valida o retorno do valor puro na primeira amostra (janela incompleta), o cálculo da média de duas amostras distintas e o comportamento da janela deslizante ao descartar amostras antigas;
- **Baseline:** confirma o acúmulo de exatamente 15 amostras válidas, a rejeição de leituras abaixo de 100 durante a fase de inicialização e a imutabilidade do baseline após sua fixação;
- **Deteção de anomalia:** testa leituras normais, buracos (teto mais distante), saliências (teto mais próximo), valores exatamente no limiar e a reconfiguração dinâmica do limiar de variação severa;


```

dados_coletados.csv > data
1 timestamp_ms;posicao_x;teto_filtrado_y;lidar_raw;encoder;aceleracao;velocidade;e_automatico;e_inspecao;confianca
2 0;0.00;0.00;0;0;50;0;0;0.10
3 98;0.00;0.00;0;0;50;0;0;0.20
4 199;0.00;0.00;0;0;50;0;0;0.30
5 300;0.00;0.00;0;0;50;0;0;0.40
6 399;0.00;0.00;0;0;50;1;0;0.50
7 499;0.00;0.00;0;0;50;1;0;0.60
8 600;0.00;70.50;143;0;0;50;1;0;0.70
9 699;0.00;145.00;147;0;0;50;1;0;0.80
10 799;0.00;147.50;145;0;0;50;1;0;0.90
11 899;0.00;144.00;144;0;0;50;1;0;1.00
12 1000;1.00;143.00;145;1;0;50;1;0;0.10
13 1099;1.00;144.00;143;1;0;50;1;0;0.20
14 1198;1.00;143.50;144;1;0;50;1;0;0.30
15 1299;1.00;142.00;148;1;0;50;1;0;0.40
16 1399;1.00;141.50;145;1;0;50;1;0;0.50
17 1500;1.00;144.50;146;1;0;50;1;0;0.60
18 1600;2.00;144.75;142;0;0;50;1;0;0.10
19 1699;2.00;143.00;145;0;0;50;1;0;0.20
20 1800;2.00;143.50;144;0;0;50;1;0;0.30
21 1900;2.00;143.00;145;0;0;50;1;0;0.40
22 1999;2.00;146.00;139;0;0;50;1;0;0.50
23 2098;3.00;144.50;144;1;0;50;1;0;0.10
24 2200;3.00;143.50;147;1;0;50;1;0;0.20
25 2300;3.00;144.50;147;1;0;50;1;0;0.30
26 2398;3.00;146.00;146;1;0;50;1;0;0.40

```

Figura 7: Log de dados do sistema.

- **Confiança:** verifica a inicialização em 10% na primeira medição, o crescimento proporcional com medições sucessivas na mesma zona, o teto em 100% e o reinício da contagem ao mudar de zona;
- **Modo de navegação:** valida a ativação e o reset dos flags `c_automatico` e `c_man`, e confirma que o modo manual tem precedência quando ambos os comandos chegam simultaneamente;
- **Clamp do PID:** verifica o corte em +100 e -100, a preservação de valores dentro dos limites e o tratamento correto dos valores exatamente nos extremos.

3.2 Testes em Python

Os testes em Python foram implementados no arquivo `src/test_sistema.py`, com estrutura equivalente ao *framework* C++ - mesma função `check` e mesmo código de saída. Foram implementados 27 testes distribuídos em 6 grupos:

- **Pool determinístico de anomalias:** confirma que a mesma semente (`random.seed(42)`) gera sequências idênticas nas duas interfaces, que os espaçamentos estão no intervalo [200, 700] px, que os tipos são válidos (*buraco* ou *protuberância*) e que sementes diferentes produzem sequências distintas;
- **Encoder:** verifica que 500 pixels geram exatamente 5 transições e que os estados alternam corretamente a cada 100 pixels;
- **Ruído do LiDAR:** valida que a média de 2000 amostras com ruído gaussiano ($\sigma = 2$) está próxima do valor real (erro $< 0,3$), que o desvio padrão calculado converge para 2,0 e que nenhuma amostra é negativa devido ao `max(0, ...)`;
- **Convergência de velocidade:** testa que a velocidade converge ao alvo em menos de 10 quadros (≈ 333 ms) tanto na aceleração ($0 \rightarrow 200$ px/s) quanto na desaceleração ($200 \rightarrow 80$ px/s), com fator de convergência 15,0 a 30 fps;

Tarefa	Tempo máximo de execução (μ s)	Período (ms)
controle_navegacao	4	80
distancia_percorrida	67	20
reconstrucao_teto	53	80
coletor_dados	346	100
inspecao_camera	8277	–

- **Confiança online:** replica os testes C++ no contexto da simulação Python, incluindo a tolerância de 1 m para manutenção de zona;
- **Baseline e detecção de anomalia:** verifica o acúmulo de 15 amostras, o cálculo correto do baseline, a não detecção de leituras normais e a detecção correta de buracos e saliências com limiar configurável.

4 Análise de Tempo de Resposta

Para a Análise em Tempo de Resposta foi necessário cronometrar as tarefas separadamente. Para isso, foi coletado o tempo máximo de execução de cada tarefa, num tempo de execução do código de 2 minutos. Assim, obteve-se os valores demonstrados na tabela 4. Em seguida, é possível calcular o fator de utilização (equação 1), obtendo um valor de 1,58% – bem mais abaixo que o valor máximo teórico de 82,8%. Assim sendo, o sistema opera a 1,9% da sua capacidade máxima de operação, isto é, com um períodos até 52 vezes maior, ainda é possível operar.

$$U = \sum_{i=0}^n \frac{C_i}{T_i} = 1,58\% \quad (1)$$

5 Compilação e Execução

A compilação do sistema é realizada por meio do arquivo **Makefile** localizado no diretório principal do projeto. Esse arquivo automatiza a compilação dos módulos desenvolvidos em C++, a ligação com as bibliotecas externas utilizadas e a geração do executável final denominado **programa**.

O projeto utiliza o compilador **g++** e o padrão C++17. Durante a compilação, também são habilitados os avisos do compilador por meio das opções **-Wall** e **-Wextra**, além do suporte à execução concorrente com a opção **-pthread**.

Os arquivos utilizados na geração do executável são:

- `main.cpp`;
- `src/sincronizacao.cpp`;
- `src/simulacao.cpp`;
- `src/mqtt_client.cpp`;
- `src/mqtt_publisher.cpp`;

- `src/mqtt_subscriber.cpp`;
- `src/mqtt_manager.cpp`.

Além do compilador C++, é necessário que estejam instaladas as bibliotecas Boost, Eclipse Paho MQTT C, Eclipse Paho MQTT C++ e `pthread`. Para a execução das interfaces gráficas e dos testes implementados em Python, também são necessários o Python 3 e a biblioteca Pygame.

O `Makefile` contém caminhos de inclusão e de ligação utilizados pelo Homebrew no macOS, como `/opt/homebrew/include` e `/opt/homebrew/lib`. Também foram incluídos os caminhos `/usr/local/include` e `/usr/local/lib`. Dessa forma, as bibliotecas necessárias devem estar instaladas em um desses diretórios ou os respectivos caminhos devem ser ajustados conforme o sistema operacional utilizado.

Inicialmente, deve-se abrir um terminal e acessar o diretório principal do projeto:

```
cd Trabalho-de-ATR
```

Para compilar todos os arquivos-fonte e gerar o executável, utiliza-se o comando:

```
make
```

Durante esse processo, cada arquivo com extensão `.cpp` é compilado separadamente, gerando os respectivos arquivos-objeto com extensão `.o`. Em seguida, esses arquivos são ligados às bibliotecas Paho MQTT e `pthread`, produzindo o executável `programa`.

Após a compilação, o sistema pode ser iniciado com:

```
make run
```

Esse comando verifica se o executável está atualizado e, em seguida, executa:

```
./programa
```

Para o funcionamento da comunicação remota, o broker Mosquitto deve estar instalado e em execução antes da inicialização do programa. Na configuração utilizada no projeto, o broker é executado localmente na porta padrão 1883.

Os testes unitários em C++ e Python podem ser executados por meio do comando:

```
make test
```

Inicialmente, esse comando compila o arquivo `src/test_sistema.cpp`, gerando o executável `test_sistema`. Em seguida, são executados os testes em C++ e, posteriormente, os testes presentes no arquivo `src/test_sistema.py` por meio do Python 3.

Para remover o executável, os arquivos-objeto e o executável dos testes, utiliza-se:

```
make clean
```

Também foi disponibilizado o comando:

```
make rebuild
```

Esse comando executa inicialmente a limpeza dos arquivos compilados e, em seguida, realiza uma nova compilação completa do projeto. Sua utilização é recomendada após alterações nos arquivos de cabeçalho ou quando for necessário garantir que todos os módulos sejam recompilados.

Assim, uma sequência completa para recompilar, testar e executar o sistema pode ser representada por:

```
make rebuild  
make test  
make run
```

Caso sejam apresentados erros relacionados às bibliotecas Paho MQTT ou Boost, deve-se verificar se elas estão corretamente instaladas e se seus diretórios de inclusão e ligação correspondem aos caminhos definidos no **Makefile**.

6 Vídeo de Demonstração

O vídeo a seguir apresenta o funcionamento básico do sistema de inspeção de túneis, incluindo a simulação física, a interface de operação remota via MQTT e alguns dos testes realizados:

https://youtu.be/aUPQ0GQ_mLQ

7 Conclusão

O presente trabalho alcançou seu propósito ao aplicar com êxito os conceitos e ferramentas de Sistemas de Tempo Real. Foi possível estabelecer a sincronização adequada entre as tarefas, emulando o comportamento de um sistema embarcado sob operação remota. Durante o desenvolvimento prático, observaram-se alguns desafios arquiteturais; em especial, a exigência de certas implementações que, embora possuam valor didático, reduziram a eficiência do código. Diante disso, elencam-se os seguintes pontos de melhoria para alterações futuras:

- **Gerenciamento de filas:** É necessário estabelecer uma periodização mais rigorosa e estruturar adequadamente a formação de filas para otimizar o acesso concorrente aos *buffers*.
- **Comunicação por *signals*:** A comunicação interprocessos (IPC) baseada exclusivamente em Memória Compartilhada é uma solução mais adequada para todo o sistema. Essa abordagem aproveitaria melhor os recursos da biblioteca Boost e eliminaria a necessidade de funções intermediárias para tratamento de sinais (como o *signal_handler*).

- **Arquitetura do *broker*:** A implementação de um *broker* em ambas as extremidades mostrou-se superdimensionada para uma rede contendo apenas duas máquinas operantes. Uma abordagem de comunicação ponto a ponto mais direta na interface simplificaria a topologia.
- **Persistência de dados:** Recomenda-se a futura integração de um banco de dados para garantir maior robustez e escalabilidade no armazenamento do histórico. Embora o modelo de persistência atual cumpra seu papel de forma satisfatória, ele apresenta limitações práticas para a manipulação analítica das informações.
- **Escalonamento por Executivo Cíclico:** Com os tempos de execução de pior caso já devidamente mapeados, a transição para uma arquitetura de escalonamento baseada em Executivo Cíclico aumentaria significativamente a previsibilidade, a eficiência temporal e o determinismo do sistema.

Em suma, a execução do projeto apresentou resultados consistentes, considerando os requisitos críticos do sistema. Entende-se que o principal obstáculo ao longo do trabalho foi a sequência de desenvolvimento, pois a mudança da etapa 1 para a etapa 2 causou bastante retrabalho. Esta dificuldade foi agravada pela incerteza ao implementar uma biblioteca muito pouco explorada em sala de aula.

8 Apêndice

8.1 Testes realizados

```

• aninhamelado@MacBook-Air-de-Ana-4 Trabalho-de-ATR % make run
./programa
[19:12:47] [PROCESS0] Processo comando_navegacao criado
Comando de navegação escreveu na memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
Controle de navegação lendo memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50

```

Figura 8: Execução inicial do programa

```

[19:12:47] [MAIN] Buffers inicializados
[19:12:47] [MAIN] Criando thread distancia_percorrida
[19:12:47] [MAIN] Criando thread inspecao_camera
[19:12:47] [MAIN] Criando thread coletor_dados
[19:12:47] [DISTANCIA] Escritor demora para segurar acesso ao BUFFER_NAVEGACAO
[19:12:47] [MAIN] Criando thread reconstrucao_teto
[19:12:47] [COLETOR] Thread inicializada

```

Figura 9: Criação das Threads

TESTE SEM DETECÇÃO DE FALHAS

```

aninhamelado@MacBook-Air-de-Ana-4 Trabalho-de-ATR % make run
./programa
[17:51:45] [PROCESSO] Processo comando_navegacao criado
Comando de navegação escreveu na memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
Controle de navegação lendo memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
[17:51:45] [MAIN] Buffers inicializados
[17:51:45] [MAIN] Criando thread distancia_percorrida
[17:51:45] [MAIN] Criando thread inspecao_camera
[17:51:45] [MAIN] Criando thread coletor_dados
[17:51:45] [MAIN] Criando thread reconstrucao_teto
[17:51:45] [MAIN] Criando thread controle_navegacao
[17:51:45] [MAIN] Esperando thread 0
[17:51:45] [COLETOR] Thread inicializada
[17:51:45] [RECONSTRUCAO] Thread inicializada
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [CONTROLE] Posição lida (navegação): 0.000000
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 10.782785
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 11.484781
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 26.471970
[17:51:45] [COLETOR] Posição lida (nível): 10.782785
[17:51:45] [COLETOR] Posição lida (nível): 11.484781
[17:51:45] [COLETOR] Posição lida (nível): 26.471970
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 39.097961
[17:51:45] [COLETOR] Posição lida (nível): 39.097961
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 68.813644
[17:51:45] [COLETOR] Posição lida (nível): 68.813644
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 64.550240
[17:51:45] [COLETOR] Posição lida (nível): 64.550240
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 58.606701
[17:51:45] [COLETOR] Posição lida (nível): 58.606701
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 59.807774
[17:51:45] [COLETOR] Posição lida (nível): 59.807774
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 63.561176
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 41.966251
[17:51:45] [COLETOR] Posição lida (nível): 63.561176
[17:51:45] [COLETOR] Posição lida (nível): 41.966251
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 53.831238
[17:51:45] [COLETOR] Posição lida (nível): 53.831238
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 69.043571

```

[17:51:45] [COLETOR] Posição lida (nível): 69.043571
 [17:51:45] [COLETOR] Buffer vazio -> aguardando dados
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 32.942486
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 31.726030
 [17:51:45] [COLETOR] Posição lida (nível): 32.942486
 [17:51:45] [COLETOR] Posição lida (nível): 31.726030
 [17:51:45] [COLETOR] Buffer vazio -> aguardando dados
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 96.737633
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 42.435276
 [17:51:45] [COLETOR] Posição lida (nível): 96.737633
 [17:51:45] [COLETOR] Posição lida (nível): 42.435276
 [17:51:45] [COLETOR] Buffer vazio -> aguardando dados
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 53.318428
 [17:51:45] [COLETOR] Posição lida (nível): 53.318428
 [17:51:45] [COLETOR] Buffer vazio -> aguardando dados
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 43.819653
 [17:51:45] [COLETOR] Posição lida (nível): 43.819653
 [17:51:45] [COLETOR] Buffer vazio -> aguardando dados
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 86.956482
 [17:51:45] [COLETOR] Posição lida (nível): 86.956482
 [17:51:45] [COLETOR] Buffer vazio -> aguardando dados
 [17:51:45] [RECONSTRUCAO] Posição escrita (nível): 38.257179
 [17:51:45] [RECONSTRUCAO] Thread finalizada
 [17:51:45] [COLETOR] Posição lida (nível): 38.257179
 [17:51:46] [DISTANCIA] Posição escrita (navegação): 28.958828
 [17:51:47] [CONTROLE] Posição lida (navegação): 0.000000
 [17:51:47] [DISTANCIA] Posição escrita (navegação): 93.278160
 [17:51:48] [DISTANCIA] Posição escrita (navegação): 5.549335
 [17:51:49] [CONTROLE] Posição lida (navegação): 5.549335
 [17:51:49] [DISTANCIA] Posição escrita (navegação): 59.613941
 [17:51:50] [DISTANCIA] Posição escrita (navegação): 5.053784
 [17:51:51] [CONTROLE] Posição lida (navegação): 59.613941
 [17:51:51] [DISTANCIA] Posição escrita (navegação): 96.715065
 [17:51:52] [DISTANCIA] Posição escrita (navegação): 24.145536
 [17:51:53] [CONTROLE] Posição lida (navegação): 5.053784
 [17:51:53] [DISTANCIA] Posição escrita (navegação): 98.272110
 [17:51:54] [DISTANCIA] Posição escrita (navegação): 64.581703
 [17:51:55] [CONTROLE] Posição lida (navegação): 96.715065
 [17:51:56] [DISTANCIA] Posição escrita (navegação): 60.326897
 [17:51:57] [DISTANCIA] Posição escrita (navegação): 94.607086
 [17:51:57] [CONTROLE] Posição lida (navegação): 24.145536
 [17:51:58] [DISTANCIA] Posição escrita (navegação): 63.498756
 [17:51:59] [DISTANCIA] Posição escrita (navegação): 72.532227
 [17:51:59] [CONTROLE] Posição lida (navegação): 98.272110
 [17:52:00] [DISTANCIA] Posição escrita (navegação): 8.689487
 [17:52:01] [DISTANCIA] Posição escrita (navegação): 33.802132
 [17:52:01] [CONTROLE] Posição lida (navegação): 64.581703
 [17:52:02] [DISTANCIA] Posição escrita (navegação): 65.608650
 [17:52:03] [DISTANCIA] Posição escrita (navegação): 84.797623
 [17:52:03] [CONTROLE] Posição lida (navegação): 60.326897
 [17:52:04] [DISTANCIA] Posição escrita (navegação): 97.202446


```

[17:52:05] [DISTANCIA] Posição escrita (navegação): 57.833843
[17:52:06] [CONTROLE] Posição lida (navegação): 94.607086
[17:52:06] [DISTANCIA] Posição escrita (navegação): 1.694813
[17:52:06] [MAIN] Thread 0 finalizada
[17:52:06] [MAIN] Esperando thread 1
[17:52:06] [MAIN] Thread 1 finalizada
[17:52:06] [MAIN] Esperando thread 2
[17:52:06] [MAIN] Thread 2 finalizada
[17:52:06] [MAIN] Esperando thread 3
[17:52:06] [MAIN] Thread 3 finalizada
[17:52:06] [MAIN] Esperando thread 4
[17:52:08] [CONTROLE] Posição lida (navegação): 63.498756
[17:52:10] [CONTROLE] Posição lida (navegação): 72.532227
[17:52:12] [CONTROLE] Posição lida (navegação): 8.689487
[17:52:14] [CONTROLE] Posição lida (navegação): 33.802132
[17:52:16] [CONTROLE] Posição lida (navegação): 65.608650
[17:52:18] [CONTROLE] Posição lida (navegação): 84.797623
[17:52:20] [CONTROLE] Posição lida (navegação): 97.202446
[17:52:22] [CONTROLE] Posição lida (navegação): 57.833843
[17:52:24] [CONTROLE] Posição lida (navegação): 1.694813
[17:52:26] [MAIN] Thread 4 finalizada
Buffer navegação: 94.6071 63.4988 72.5322 8.68949 33.8021 65.6087 84.7976
↪ 97.2024 57.8338 1.69481
Buffer nível: 53.8312 69.0436 32.9425 31.726 96.7376 42.4353 53.3184 43.8197
↪ 86.9565 38.2572
[17:52:26] [MAIN] Execução finalizada

```

TESTE COM UMA FALHA DETECTADA

```

aninhamelado@MacBook-Air-de-Ana-4 Trabalho-de-ATR % make run
./programa
[18:15:56] [PROCESSO] Processo comando_navegacao criado
Comando de navegação escreveu na memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
Controle de navegação lendo memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
[18:15:56] [MAIN] Buffers inicializados
[18:15:56] [MAIN] Criando thread distancia_percorrida
[18:15:56] [MAIN] Criando thread inspecao_camera
[18:15:56] [MAIN] Criando thread coletor_dados
[18:15:56] [MAIN] Criando thread reconstrucao_teto
[18:15:56] [COLETOR] Thread inicializada
[18:15:56] [RECONSTRUCAO] Thread inicializada
[18:15:56] [COLETOR] Buffer vazio -> aguardando dados
[18:15:56] [MAIN] Criando thread controle_navegacao
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 74.495483
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 42.898727
[18:15:56] [CONTROLE] Posição lida (navegação): 0.000000
[18:15:56] [MAIN] Esperando thread 0

```

[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 34.575493
 [18:15:56] [COLETOR] Posição lida (nível): 74.495483
 [18:15:56] [COLETOR] Posição lida (nível): 42.898727
 [18:15:56] [COLETOR] Posição lida (nível): 34.575493
 [18:15:56] [COLETOR] Buffer vazio -> aguardando dados
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 98.192505
 [18:15:56] [COLETOR] Posição lida (nível): 98.192505
 [18:15:56] [COLETOR] Buffer vazio -> aguardando dados
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 15.192101
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 73.814850
 [18:15:56] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:15:56] [COLETOR] Posição lida (nível): 15.192101
 [18:15:56] [COLETOR] Posição lida (nível): 73.814850
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 97.350990
 [18:15:56] [CAMERA] Inspeção iniciada
 [18:15:56] [CAMERA] Inspeção finalizada
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 55.099091
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 48.265957
 [18:15:56] [COLETOR] Posição lida (nível): 97.350990
 [18:15:56] [COLETOR] Posição lida (nível): 55.099091
 [18:15:56] [COLETOR] Posição lida (nível): 48.265957
 [18:15:56] [COLETOR] Buffer vazio -> aguardando dados
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 30.761057
 [18:15:56] [COLETOR] Posição lida (nível): 30.761057
 [18:15:56] [COLETOR] Buffer vazio -> aguardando dados
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 49.804440
 [18:15:56] [COLETOR] Posição lida (nível): 49.804440
 [18:15:56] [COLETOR] Buffer vazio -> aguardando dados
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 46.795132
 [18:15:56] [COLETOR] Posição lida (nível): 46.795132
 [18:15:56] [COLETOR] Buffer vazio -> aguardando dados
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 12.473819
 [18:15:56] [COLETOR] Posição lida (nível): 12.473819
 [18:15:56] [COLETOR] Buffer vazio -> aguardando dados
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 68.368179
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 34.957470
 [18:15:56] [COLETOR] Posição lida (nível): 68.368179
 [18:15:56] [COLETOR] Posição lida (nível): 34.957470
 [18:15:56] [COLETOR] Buffer vazio -> aguardando dados
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 90.110184
 [18:15:56] [COLETOR] Posição lida (nível): 90.110184
 [18:15:56] [COLETOR] Buffer vazio -> aguardando dados
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 39.241226
 [18:15:56] [COLETOR] Posição lida (nível): 39.241226
 [18:15:56] [COLETOR] Buffer vazio -> aguardando dados
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 31.790064
 [18:15:56] [COLETOR] Posição lida (nível): 31.790064
 [18:15:56] [COLETOR] Buffer vazio -> aguardando dados
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 93.427567
 [18:15:56] [RECONSTRUCAO] Posição escrita (nível): 61.848675

```

[18:15:56] [RECONSTRUCAO] Thread finalizada
[18:15:56] [COLETOR] Posição lida (nível): 93.427567
[18:15:56] [COLETOR] Posição lida (nível): 61.848675
[18:15:57] [DISTANCIA] Posição escrita (navegação): 62.380260
[18:15:58] [CONTROLE] Posição lida (navegação): 0.000000
[18:15:58] [DISTANCIA] Posição escrita (navegação): 51.585777
[18:15:59] [DISTANCIA] Posição escrita (navegação): 62.972610
[18:16:00] [CONTROLE] Posição lida (navegação): 62.972610
[18:16:00] [DISTANCIA] Posição escrita (navegação): 65.160225
[18:16:01] [DISTANCIA] Posição escrita (navegação): 31.197983
[18:16:02] [CONTROLE] Posição lida (navegação): 65.160225
[18:16:02] [DISTANCIA] Posição escrita (navegação): 45.259418
[18:16:03] [DISTANCIA] Posição escrita (navegação): 81.480118
[18:16:04] [CONTROLE] Posição lida (navegação): 31.197983
[18:16:04] [DISTANCIA] Posição escrita (navegação): 19.269875
[18:16:05] [DISTANCIA] Posição escrita (navegação): 17.381445
[18:16:06] [CONTROLE] Posição lida (navegação): 45.259418
[18:16:06] [DISTANCIA] Posição escrita (navegação): 43.342255
[18:16:07] [DISTANCIA] Posição escrita (navegação): 1.306038
[18:16:08] [CONTROLE] Posição lida (navegação): 81.480118
[18:16:08] [DISTANCIA] Posição escrita (navegação): 12.224937
[18:16:09] [DISTANCIA] Posição escrita (navegação): 75.290260
[18:16:10] [CONTROLE] Posição lida (navegação): 19.269875
[18:16:10] [DISTANCIA] Posição escrita (navegação): 55.725075
[18:16:11] [DISTANCIA] Posição escrita (navegação): 83.067741
[18:16:12] [CONTROLE] Posição lida (navegação): 17.381445
[18:16:12] [DISTANCIA] Posição escrita (navegação): 20.277987
[18:16:13] [DISTANCIA] Posição escrita (navegação): 60.572338
[18:16:14] [CONTROLE] Posição lida (navegação): 43.342255
[18:16:14] [DISTANCIA] Posição escrita (navegação): 44.572971
[18:16:15] [DISTANCIA] Posição escrita (navegação): 38.374176
[18:16:16] [CONTROLE] Posição lida (navegação): 1.306038
[18:16:16] [DISTANCIA] Posição escrita (navegação): 34.950138
[18:16:16] [MAIN] Thread 0 finalizada
[18:16:16] [MAIN] Esperando thread 1
[18:16:16] [MAIN] Thread 1 finalizada
[18:16:16] [MAIN] Esperando thread 2
[18:16:16] [MAIN] Thread 2 finalizada
[18:16:16] [MAIN] Esperando thread 3
[18:16:16] [MAIN] Thread 3 finalizada
[18:16:16] [MAIN] Esperando thread 4
[18:16:18] [CONTROLE] Posição lida (navegação): 12.224937
[18:16:20] [CONTROLE] Posição lida (navegação): 75.290260
[18:16:22] [CONTROLE] Posição lida (navegação): 55.725075
[18:16:24] [CONTROLE] Posição lida (navegação): 83.067741
[18:16:26] [CONTROLE] Posição lida (navegação): 20.277987
[18:16:28] [CONTROLE] Posição lida (navegação): 60.572338
[18:16:30] [CONTROLE] Posição lida (navegação): 44.572971
[18:16:32] [CONTROLE] Posição lida (navegação): 38.374176
[18:16:34] [CONTROLE] Posição lida (navegação): 34.950138
[18:16:36] [MAIN] Thread 4 finalizada

```

```
Buffer navegação: 1.30604 12.2249 75.2903 55.7251 83.0677 20.278 60.5723
↳ 44.573 38.3742 34.9501
Buffer nível: 49.8044 46.7951 12.4738 68.3682 34.9575 90.1102 39.2412 31.7901
↳ 93.4276 61.8487
[18:16:36] [MAIN] Execução finalizada
```

TESTE COM MÚLTIPLAS FALHAS

```
aninhamelado@MacBook-Air-de-Ana-4 Trabalho-de-ATR % make run
./programa
[18:14:34] [PROCESSO] Processo comando_navegacao criado
Comando de navegação escreveu na memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
Controle de navegação lendo memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
[18:14:34] [MAIN] Buffers inicializados
[18:14:34] [MAIN] Criando thread distancia_percorrida
[18:14:34] [MAIN] Criando thread inspecao_camera
[18:14:34] [MAIN] Criando thread coletor_dados
[18:14:34] [MAIN] Criando thread reconstrucao_teto
[18:14:34] [COLETOR] Thread inicializada
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [RECONSTRUCAO] Thread inicializada
[18:14:34] [MAIN] Criando thread controle_navegacao
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 11.893382
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [MAIN] Esperando thread 0
[18:14:34] [COLETOR] Posição lida (nível): 11.893382
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CONTROLE] Posição lida (navegação): 0.000000
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 33.125107
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 33.125107
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 70.220352
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 70.220352
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 38.717987
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
```

[18:14:34] [COLETOR] Posição lida (nível): 38.717987
 [18:14:34] [CAMERA] Inspeção iniciada
 [18:14:34] [CAMERA] Inspeção finalizada
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados
 [18:14:34] [RECONSTRUCAO] Posição escrita (nível): 89.701645
 [18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:14:34] [COLETOR] Posição lida (nível): 89.701645
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados
 [18:14:34] [RECONSTRUCAO] Posição escrita (nível): 17.111982
 [18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:14:34] [COLETOR] Posição lida (nível): 17.111982
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados
 [18:14:34] [CAMERA] Inspeção iniciada
 [18:14:34] [CAMERA] Inspeção finalizada
 [18:14:34] [CAMERA] Inspeção iniciada
 [18:14:34] [CAMERA] Inspeção finalizada
 [18:14:34] [RECONSTRUCAO] Posição escrita (nível): 85.353203
 [18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:14:34] [COLETOR] Posição lida (nível): 85.353203
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados
 [18:14:34] [RECONSTRUCAO] Posição escrita (nível): 42.381588
 [18:14:34] [CAMERA] Inspeção iniciada
 [18:14:34] [CAMERA] Inspeção finalizada
 [18:14:34] [COLETOR] Posição lida (nível): 42.381588
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados
 [18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:14:34] [CAMERA] Inspeção iniciada
 [18:14:34] [CAMERA] Inspeção finalizada
 [18:14:34] [RECONSTRUCAO] Posição escrita (nível): 18.459126
 [18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:14:34] [COLETOR] Posição lida (nível): 18.459126
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados
 [18:14:34] [CAMERA] Inspeção iniciada
 [18:14:34] [CAMERA] Inspeção finalizada
 [18:14:34] [RECONSTRUCAO] Posição escrita (nível): 78.904755
 [18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:14:34] [COLETOR] Posição lida (nível): 78.904755
 [18:14:34] [CAMERA] Inspeção iniciada
 [18:14:34] [CAMERA] Inspeção finalizada
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados
 [18:14:34] [RECONSTRUCAO] Posição escrita (nível): 46.468765
 [18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:14:34] [COLETOR] Posição lida (nível): 46.468765
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados

```

[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 68.231094
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 68.231094
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 70.282280
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 70.282280
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 47.991817
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 47.991817
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 55.565323
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 55.565323
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 76.516632
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 76.516632
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 59.348728
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 59.348728
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 8.146796
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 8.146796
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados

```

```

[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 68.844254
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 68.844254
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 26.924643
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 26.924643
[18:14:34] [RECONSTRUCAO] Thread finalizada
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:35] [DISTANCIA] Posição escrita (navegação): 93.736534
[18:14:36] [DISTANCIA] Escritor aguardando acesso ao buffer navegacao
[18:14:36] [CONTROLE] Posição lida (navegação): 0.000000
[18:14:36] [DISTANCIA] Escritor retomou execução
[18:14:36] [DISTANCIA] Posição escrita (navegação): 80.958572
[18:14:37] [DISTANCIA] Posição escrita (navegação): 35.029560
[18:14:38] [CONTROLE] Posição lida (navegação): 35.029560
[18:14:38] [DISTANCIA] Posição escrita (navegação): 91.344322
[18:14:39] [DISTANCIA] Posição escrita (navegação): 96.001343
[18:14:40] [CONTROLE] Posição lida (navegação): 91.344322
[18:14:40] [DISTANCIA] Posição escrita (navegação): 11.806067
[18:14:41] [DISTANCIA] Posição escrita (navegação): 73.489479
[18:14:42] [CONTROLE] Posição lida (navegação): 96.001343
[18:14:42] [DISTANCIA] Posição escrita (navegação): 10.448226
[18:14:43] [DISTANCIA] Posição escrita (navegação): 62.412590
[18:14:44] [CONTROLE] Posição lida (navegação): 11.806067
[18:14:44] [DISTANCIA] Posição escrita (navegação): 64.524155
[18:14:45] [DISTANCIA] Posição escrita (navegação): 19.370922
[18:14:46] [CONTROLE] Posição lida (navegação): 73.489479
[18:14:46] [DISTANCIA] Posição escrita (navegação): 46.210743
[18:14:47] [DISTANCIA] Posição escrita (navegação): 97.758255
[18:14:48] [CONTROLE] Posição lida (navegação): 10.448226
[18:14:48] [DISTANCIA] Posição escrita (navegação): 80.271263
[18:14:49] [DISTANCIA] Posição escrita (navegação): 34.450363
[18:14:50] [CONTROLE] Posição lida (navegação): 62.412590
[18:14:50] [DISTANCIA] Posição escrita (navegação): 30.559107
[18:14:51] [DISTANCIA] Posição escrita (navegação): 68.277695
[18:14:52] [CONTROLE] Posição lida (navegação): 64.524155
[18:14:52] [DISTANCIA] Posição escrita (navegação): 99.241768
[18:14:53] [DISTANCIA] Posição escrita (navegação): 39.216423
[18:14:54] [CONTROLE] Posição lida (navegação): 19.370922
[18:14:54] [DISTANCIA] Posição escrita (navegação): 28.282803
[18:14:54] [MAIN] Thread 0 finalizada
[18:14:54] [MAIN] Esperando thread 1
[18:14:54] [MAIN] Thread 1 finalizada
[18:14:54] [MAIN] Esperando thread 2
[18:14:54] [MAIN] Thread 2 finalizada

```

```

[18:14:54] [MAIN] Esperando thread 3
[18:14:54] [MAIN] Thread 3 finalizada
[18:14:54] [MAIN] Esperando thread 4
[18:14:56] [CONTROLE] Posição lida (navegação): 46.210743
[18:14:58] [CONTROLE] Posição lida (navegação): 97.758255
[18:15:00] [CONTROLE] Posição lida (navegação): 80.271263
[18:15:02] [CONTROLE] Posição lida (navegação): 34.450363
[18:15:04] [CONTROLE] Posição lida (navegação): 30.559107
[18:15:06] [CONTROLE] Posição lida (navegação): 68.277695
[18:15:08] [CONTROLE] Posição lida (navegação): 99.241768
[18:15:10] [CONTROLE] Posição lida (navegação): 39.216423
[18:15:12] [CONTROLE] Posição lida (navegação): 28.282803
[18:15:14] [MAIN] Thread 4 finalizada
Buffer navegação: 19.3709 46.2107 97.7583 80.2713 34.4504 30.5591 68.2777
↪ 99.2418 39.2164 28.2828
Buffer nível: 46.4688 68.2311 70.2823 47.9918 55.5653 76.5166 59.3487 8.1468
↪ 68.8443 26.9246
[18:15:14] [MAIN] Execução finalizada

```

```

[19:06:52] [COLETOR] Buffer de nível vazio -> aguardando dados
[19:06:52] [DISTANCIA] Posição escrita (navegação): 24.064884
[19:06:52] [RECONSTRUCAO] Thread inicializada
[19:06:52] [MAIN] Criando thread controle_navegacao
[19:06:52] [RECONSTRUCAO] Posição escrita (nível): 149.649765 | Ocupação: 1/10
[19:06:52] [DISTANCIA] Posição escrita (navegação): 2.887431
[19:06:52] [CONTROLE] Leitor demora para segurar acesso ao BUFFER_NAVEGACAO
[19:06:52] [RECONSTRUCAO] Posição escrita (nível): 86.107315 | Ocupação: 2/10
[19:06:52] [MAIN] Esperando thread 0
[19:06:52] [COLETOR] Posição lida (nível): 149.649765
[19:06:52] [COLETOR] Posição lida (nível): 86.107315
[19:06:52] [COLETOR] Buffer de nível vazio -> aguardando dados

```

Figura 10: Teste de Buffer de Nível Vazio


```

[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 112.025932 | Ocupação: 1/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 160.867523 | Ocupação: 2/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 122.139771 | Ocupação: 3/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 86.595070 | Ocupação: 4/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 146.985657 | Ocupação: 5/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 39.833908 | Ocupação: 6/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 103.447708 | Ocupação: 7/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 92.925583 | Ocupação: 8/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 125.545197 | Ocupação: 9/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 124.471046 | Ocupação: 10/10
[19:13:01] [RECONSTRUCAO] Buffer de nível cheio -> produtor aguardando espaço
[19:13:01] [CONTROLE] Leitor retomou execução
[19:13:01] [DISTANCIA] Posição escrita (navegação): 71.950806
[19:13:01] [DISTANCIA] Escritor aguardando acesso ao buffer navegacao
[19:13:01] [COLETOR] Posição lida (nível): 112.025932
[19:13:01] [COLETOR] Posição lida (nível): 160.867523
[19:13:01] [COLETOR] Posição lida (nível): 122.139771
[19:13:01] [COLETOR] Posição lida (nível): 86.595070
[19:13:01] [COLETOR] Posição lida (nível): 146.985657
[19:13:01] [COLETOR] Posição lida (nível): 39.833908
[19:13:01] [COLETOR] Posição lida (nível): 103.447708
[19:13:01] [COLETOR] Posição lida (nível): 92.925583
[19:13:01] [COLETOR] Posição lida (nível): 125.545197
[19:13:01] [COLETOR] Posição lida (nível): 124.471046
[19:13:01] [COLETOR] Buffer de nível vazio -> aguardando dados
[19:13:01] [RECONSTRUCAO] Espaço do buffer de nível liberado -> produtor retomou execução
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 52.224766 | Ocupação: 1/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 112.025932 | Ocupação: 2/10

```

Figura 11: Teste de Buffer de Nível Cheio

```

[19:12:47] [DISTANCIA] Escritor demora para segurar acesso ao BUFFER_NAVEGACAO
[19:12:47] [MAIN] Criando thread reconstrucao_teto
[19:12:47] [COLETOR] Thread inicializada
[19:12:47] [COLETOR] Buffer de nível vazio -> aguardando dados
[19:12:47] [RECONSTRUCAO] Thread inicializada
[19:12:47] [RECONSTRUCAO] Leitor aguardando acesso ao buffer navegacao
[19:12:47] [MAIN] Criando thread controle_navegacao
[19:12:47] [MAIN] Esperando thread 0
[19:12:47] [CONTROLE] Leitor aguardando acesso ao buffer navegacao
[19:12:48] [DISTANCIA] Posição escrita (navegação): 73.074905
[19:12:48] [RECONSTRUCAO] Leitor retomou execução
[19:12:48] [CONTROLE] Leitor retomou execução

```

Figura 12: Teste de Bloqueio de Escritores

```

[19:06:52] [MAIN] Criando thread controle_navegacao
[19:06:52] [RECONSTRUCAO] Posição escrita (nível): 149.649765 | Ocupação: 1/10
[19:06:52] [DISTANCIA] Posição escrita (navegação): 2.887431
[19:06:52] [CONTROLE] Leitor demora para segurar acesso ao BUFFER_NAVEGACAO
[19:06:52] [RECONSTRUCAO] Posição escrita (nível): 86.107315 | Ocupação: 2/10
[19:06:52] [MAIN] Esperando thread 0
[19:06:52] [COLETOR] Posição lida (nível): 149.649765
[19:06:52] [COLETOR] Posição lida (nível): 86.107315
[19:06:52] [COLETOR] Buffer de nível vazio -> aguardando dados
[19:06:52] [DISTANCIA] Escritor aguardando acesso ao buffer navegacao
[19:06:53] [CONTROLE] Posição lida (navegação): 53.040310
[19:06:53] [DISTANCIA] Escritor retomou execução
[19:06:53] [DISTANCIA] Posição escrita (navegação): 99.067177
[19:06:53] [DISTANCIA] Escritor aguardando acesso ao buffer navegacao
[19:06:53] [CONTROLE] Leitor demora para segurar acesso ao BUFFER_NAVEGACAO

```

Figura 13: Teste de Bloqueio de Leitores

```

[19:13:18] [MAIN] Thread 0 finalizada
[19:13:18] [MAIN] Esperando thread 1
[19:13:18] [MAIN] Thread 1 finalizada
[19:13:18] [MAIN] Esperando thread 2
[19:13:18] [MAIN] Thread 2 finalizada
[19:13:18] [MAIN] Esperando thread 3
[19:13:18] [MAIN] Thread 3 finalizada
[19:13:18] [MAIN] Esperando thread 4
[19:13:19] [CONTROLE] Posição lida (navegação): 97.285744
[19:13:20] [CONTROLE] Posição lida (navegação): 12.477365
[19:13:21] [CONTROLE] Posição lida (navegação): 46.973194
[19:13:22] [CONTROLE] Posição lida (navegação): 20.479900
[19:13:23] [CONTROLE] Posição lida (navegação): 82.494011
[19:13:24] [CONTROLE] Posição lida (navegação): 25.792789
[19:13:24] [MAIN] Thread 4 finalizada
Buffer navegação: 14.1857 37.1402 80.635 85.4882 97.2857 12.4774 46.9732 20.4799 82.494 25.7928
Buffer nível: 146.986 39.8339 103.448 92.9256 125.545 124.471 52.2248 108.817 163.728 65.7006
[19:13:24] [MAIN] Execução finalizada

```

Figura 14: Teste de Finalização das Threads

8.2 includes.hpp

```

1  #pragma once
2
3  #include <iostream>
4  #include <vector>
5  #include <chrono>
6  #include <cstdlib>
7  #include <cstring>
8  #include <csignal>
9  #include <random>
10
11 #include <thread>
12 #include <atomic>
13 #include <mutex>
14 #include <condition_variable>
15 #include <semaphore>
16 #include <pthread.h>
17 #include <iomanip>
18
19 #include <unistd.h>
20 #include <sys/ipc.h>
21 #include <sys/shm.h>
22 #include <sys/wait.h>
23
24 #include <functional>
25 #include <boost/asio.hpp>
26 #include <sys/wait.h>
27
28 #include <boost/interprocess/file_mapping.hpp>
29 #include <boost/interprocess/mapped_region.hpp>
30 #include <fstream>
31 #include <cstdio>
32 #include <algorithm>
33 #include <cmath>

```

```

34
35 // MQTT
36 #include "mqtt_client.hpp"
37 #include "mqtt_publisher.hpp"
38 #include "mqtt_subscriber.hpp"
39 #include "mqtt_manager.hpp"
40 #include "mqtt_config.hpp"

```

8.3 main.cpp

```

1  #include "include/includes.hpp"
2  #include "include/shared_memory.hpp"
3  #include "include/sincronizacao.hpp"
4  #include "include/simulacao.hpp"
5
6  int main (){
7
8      using namespace boost::interprocess;
9
10     simulacao();
11
12     std::remove(SHM_FILE); // remove a memória compartilhada caso ela já
        ↳ exista
13
14     {
15         std::filebuf fbuf;
16         fbuf.open(SHM_FILE, std::ios_base::in | std::ios_base::out |
        ↳ std::ios_base::trunc | std::ios_base::binary);
17
18         if(!fbuf.is_open()){
19             std::cerr << "Erro ao criar arquivo de memória compartilhada" <<
        ↳ std::endl;
20             return 1;
21         }
22
23         fbuf.pubseekoff(sizeof(MemoriaCompartilhada)-1, std::ios_base::beg);
24         fbuf.sputc(0);
25         fbuf.close();
26     } // cria arquivo que será mapeado na memória
27
28     std::cout << "Arquivo de memória criado em: " << SHM_FILE << std::endl;
29     std::cout << "Tamanho da struct C++: " << sizeof(MemoriaCompartilhada) << "
        ↳ bytes" << std::endl;
30
31     file_mapping shm_file(SHM_FILE, read_write);
32     mapped_region region(shm_file, read_write); // mapeia o arquivo no espaço
        ↳ de memória
33
34     MemoriaCompartilhada* shm =
        ↳ static_cast<MemoriaCompartilhada*>(region.get_address());
35

```

```

36 // Inicializa os valores da memória compartilhada
37 shm->i_encoder = false;
38 shm->i_lidar = 0;
39
40 shm->o_liga_camera = false;
41 shm->o_aceleracao = 0;
42
43 shm->e_inspecao = false;
44 shm->e_automatico = false;
45
46 shm->c_automatico = false;
47 shm->c_man = false;
48 shm->j_sp_velocidade = 0;
49
50 shm->c_encerrar = false;
51
52 shm->variacao_severa = 10.0f; // limiar padrão (configurável via MQTT)
53 shm->posicao_x = 0.0f;
54 shm->perfil_y = 0.0f;
55 shm->perfil_confianca = 0.0f;
56 shm->perfil_novo = false;
57
58
59 // CRIAÇÃO DE PROCESSOS (FORK)
60
61 // PROCESSO 1: SIMULAÇÃO (interface.py usa shared memory)
62 pid_t pid_interface = fork();
63 if (pid_interface == 0) {
64     log_message("PROCESSO", "Processo de simulação Pygame criado");
65     execlp("python3", "python3", "src/interface.py", nullptr);
66     perror("Erro ao executar interface.py");
67     exit(1);
68 } else if (pid_interface < 0) {
69     perror("Erro ao criar processo da interface");
70     exit(1);
71 }
72
73 //PROCESSO 2: OPERAÇÃO REMOTA (interface_mqtt.py usa MQTT)
74 pid_t pid_remoto = fork();
75 if (pid_remoto == 0) {
76     log_message("PROCESSO", "Processo de operação remota MQTT criado");
77     execlp("python3", "python3", "src/interface_mqtt.py", nullptr);
78     perror("Erro ao executar interface_mqtt.py");
79     exit(1);
80 } else if (pid_remoto < 0) {
81     perror("Erro ao criar processo de operação remota");
82     exit(1);
83 }
84
85
86 // PROCESSO 2: COMANDO (FILHO) E CONTROLE (PAI)

```

```

87  pid_t pid_controle = fork();
88
89  if (pid_controle == 0) {
90      // BLOCO DO FILHO: COMANDO DE NAVEGAÇÃO
91      log_message("PROCESSO", "Processo comando_navegacao criado");
92
93      file_mapping shm_child(SHM_FILE, read_write); // abre a memória
          ↪ compartilhada criada
94
95      mapped_region region_child(shm_child, read_write);
96      MemoriaCompartilhada* shm_filho =
          ↪ static_cast<MemoriaCompartilhada*>(region_child.get_address());
97
98      // Instanciar o Asio APENAS para o filho
99      boost::asio::io_context io_filho;
100     boost::asio::io_context::strand strand_filho(io_filho);
101
102     tempo_tarefa t_comando(io_filho, milisegundos(500));
103     t_comando.async_wait(boost::asio::bind_executor(strand_filho,
104         std::bind(comando_navegacao, std::placeholders::_1, &t_comando,
          ↪ &strand_filho, shm)));
105
106     shm->c_automatico = true;
107     shm->j_sp_velocidade = 50;
108
109     std::cout << "Comando de navegação escreveu na memória compartilhada:" <<
          ↪ std::endl;
110     std::cout << "c_automatico = " << shm_filho->c_automatico <<
          ↪ std::endl;
111     std::cout << "j_sp_velocidade = " << shm_filho->j_sp_velocidade <<
          ↪ std::endl;
112
113     // Iniciar o laço de eventos do filho
114     io_filho.run();
115
116     exit(0);
117 }
118 else if (pid_controle < 0) {
119     perror("Erro ao criar processo de comando\n");
120     exit(1);
121 }
122 else {
123     // BLOCO DO PAI: CONTROLE DE NAVEGAÇÃO E SENSORIAMENTO
124     log_message("PROCESSO", "Processo de controle de navegação iniciado");
125
126     std::mutex shm_mutex;
127
128     MQTTManager mqtt_manager(shm, shm_mutex);
129
130     log_message("MAIN", "Inicializando MQTT Manager...");
131     if (mqtt_manager.initialize()) {

```

```

132     log_message("MAIN", "MQTT Manager inicializado com sucesso");
133 } else {
134     log_message("MAIN", "Falha ao inicializar MQTT Manager - continuando
        ↳ sem MQTT");
135 }
136
137 boost::asio::io_context io_pai;
138
139 boost::asio::io_context::strand strand_navegacao(io_pai); // Para dist e
        ↳ controle
140 boost::asio::io_context::strand strand_processamento(io_pai); // Para
        ↳ reconstrução e dados
141 boost::asio::io_context::strand strand_camera(io_pai); // Para câmera
        ↳ isolada
142
143 struct VarCondSinc sinc;
144
145 // DEFINIÇÃO DOS TEMPORIZADORES
146 tempo_tarefa t1_dist(io_pai, milisegundos(PERODO_DISTANCIA));
147 tempo_tarefa t2_cam(io_pai, milisegundos(PERODO_CAMERA));
148 tempo_tarefa t3_col(io_pai, milisegundos(PERODO_COLETOR));
149 tempo_tarefa t4_rec(io_pai, milisegundos(PERODO_RECONSTRUCAO));
150 tempo_tarefa t5_con(io_pai, milisegundos(PERODO_CONTROLE));
151
152 //CRIAÇÃO DE SIGNAL
153 boost::asio::signal_set sinais (io_pai);
154 sinais.add(SIGUSR1);
155
156 // AGENDAMENTO DAS TAREFAS (ASYNC_WAIT)
157 log_message("MAIN", "Agendando tarefas assíncronas...");
158
159 // Associando à Strand de Navegação (Crítica)
160 t1_dist.async_wait(boost::asio::bind_executor(strand_navegacao,
161     std::bind(distancia_percorrida, std::placeholders::_1, &t1_dist,
        ↳ &strand_navegacao, shm, std::ref(sinc))));
162
163 t5_con.async_wait(boost::asio::bind_executor(strand_navegacao,
164     std::bind(controle_navegacao, std::placeholders::_1, &t5_con,
        ↳ &strand_navegacao, shm, std::ref(sinc))));
165
166 // Associando à Strand de Processamento (Menos Crítica)
167 t4_rec.async_wait(boost::asio::bind_executor(strand_processamento,
168     std::bind(reconstrucao_teto, std::placeholders::_1, &t4_rec,
        ↳ &strand_processamento, shm, std::ref(sinc))));
169
170 t3_col.async_wait(boost::asio::bind_executor(strand_processamento,
171     std::bind(coletor_dados, std::placeholders::_1, &t3_col,
        ↳ &strand_processamento, shm, std::ref(sinc))));
172
173 sinais.async_wait(

```

```

174         std::bind(handler_signal, std::placeholders::_1, std::placeholders::_2,
    ↪      &t2_cam, &strand_camera,
175             std::ref(sinc.mutex_camera), shm, &sinais));
176
177     // EXECUÇÃO (THREAD POOL)
178     // Reativar o Pool de Threads. Com 4 threads, as nossas 3 Strands podem
    ↪ rodar
179     // simultaneamente em núcleos reais do processador.
180
181     while (!shm->c_automatico) {
182         std::this_thread::sleep_for(std::chrono::milliseconds(5));
183     }
184
185     int num_threads = 4;
186     std::vector<std::thread> thread_pool;
187     log_message("MAIN", "Iniciando Thread Pool do Boost.Asio com " +
    ↪      std::to_string(num_threads) + " threads");
188
189     for (int i = 0; i < num_threads; ++i) {
190         thread_pool.emplace_back([&io_pai]() { io_pai.run(); });
191     }
192
193     std::cout << "Sistema rodando. Feche a janela de simulação para
    ↪      encerrar.\n";
194
195     // Aguarda até que a interface de simulação sinalize o encerramento (janela
    ↪ fechada)
196     while (!shm->c_encerrar) {
197         std::this_thread::sleep_for(std::chrono::milliseconds(200));
198     }
199
200     std::cout << "Sinal de encerramento recebido. Desligando tarefas...\n";
201
202     io_pai.stop();
203
204     //SINCRONIZAÇÃO FINAL
205     for (auto& t : thread_pool) {
206         if (t.joinable()) {
207             std::cout << "Esperando join da thread...\n";
208             t.join();
209         }
210     }
211
212     std::cout << "Thread pool encerrada.\n";
213
214     // Espera o processo filho acabar
215     waitpid(pid_controle, nullptr, 0);
216
217     // Finalizar MQTT Manager
218     log_message("MAIN", "Finalizando MQTT Manager...");
219     mqtt_manager.shutdown();

```



```

220     log_message("MAIN", "MQTT Manager finalizado");
221
222     log_message("MAIN", "Execução finalizada. Limpando memória.");
223
224     // Desanexa memória no pai
225     std::remove(SHM_FILE);
226 }
227 }

```

8.4 sincronizacao.cpp

```

1  #include "sincronizacao.hpp"
2
3  #define ELEMENTOS_BUFFERS 10
4
5  //sincronização
6  std::condition_variable camera;
7
8  //teste para finalizar a camera
9  bool finalizar_camera = false;
10 int eventos_camera = 0;
11
12 //Variaveis de condição buffers
13 std::condition_variable leitura_buffer_navegacao;
14 std::condition_variable escrita_buffer_navegacao;
15
16 int AR_NAVEGACAO = 0;
17 int WR_NAVEGACAO = 0;
18 int AW_NAVEGACAO = 0;
19 int WW_NAVEGACAO = 0;
20
21 std::condition_variable leitura_buffer_nivel;
22 std::condition_variable escrita_buffer_nivel;
23 int dados_nivel = 0;
24
25 //Funções auxiliares de debug
26 std::mutex mutex_log;
27 int miss[4] = {0, 0, 0, 0};
28 int executado[4] = {0, 0, 0, 0};
29
30
31 // Estimativa de velocidade compartilhada entre distancia_percorrida e
   ↪ controle_navegacao.
32 // Ambas as tarefas rodam na mesma strand (strand_navegacao), portanto sem
   ↪ corrida de dados.
33 static float s_vel_encoder_ms = 0.0f;
34 static auto s_t_ultimo_encoder = std::chrono::steady_clock::now();
35 static bool s_encoder_ativo = false;
36
37 void log_message(const std::string& thread, const std::string& mensagem) {
38     std::lock_guard<std::mutex> lock(mutex_log);

```



```

39     auto agora = std::chrono::system_clock::now();
40     auto tempo = std::chrono::system_clock::to_time_t(agora);
41     std::cout << "[" << std::put_time(std::localtime(&tempo), "%H:%M:%S") << "]"
    ↪     "
42         << "[" << thread << "]" " << mensagem << std::endl;
43 }
44
45 float numero_aleatorio_debugg() {
46     std::random_device rd;
47     std::mt19937 gen(rd());
48     std::uniform_real_distribution<float> dis(1.0f, 100.0f);
49     return dis(gen);
50 }
51
52
53 // FUNÇÃO AUXILIAR DE TEMPO REAL
54 void reagendar_tarefa(boost::asio::steady_timer* t, int periodo_us, const
    ↪     std::string& nome_tarefa) {
55     auto agora = std::chrono::steady_clock::now();
56     auto atraso = std::chrono::duration_cast<std::chrono::milliseconds>(agora -
    ↪     t->expiry()).count();
57     if (atraso > 5000) {
58         log_message(nome_tarefa, "ALERTA: Deadline violado! Atraso de " +
    ↪         std::to_string(atraso) + " ms");
59     }
60
61     auto proximo_ciclo = t->expiry() +
    ↪     boost::asio::chrono::milliseconds(periodo_us);
62
63     if (proximo_ciclo < agora) {
64         proximo_ciclo = agora + boost::asio::chrono::milliseconds(periodo_us);
65     }
66
67     t->expires_at(proximo_ciclo);
68 }
69
70 void handler_signal(const boost::system::error_code& error, int signal_number,
71     boost::asio::steady_timer* t,
    ↪     boost::asio::io_context::strand* strand_camera,
72     std::mutex& mtx, MemoriaCompartilhada* shm,
    ↪     boost::asio::signal_set* sinais) {
73
74     if(error) return;
75
76     if (signal_number == SIGUSR1) {
77         eventos_camera++;
78
79         boost::asio::post(*strand_camera, std::bind(inspecao_camera,
80             boost::system::error_code(), t, strand_camera,
    ↪             std::ref(mtx), shm));
81     }

```

```

82
83 //REAGENDAMENTO
84
85 sinais->async_wait(std::bind(handler_signal, std::placeholders::_1,
86                               ↪ std::placeholders::_2,
87                               t, strand_camera, std::ref(mtx), shm,
88                               ↪ sinais));
89
90 // TAREFAS ASSÍNCRONAS DO SISTEMA
91 void comando_navegacao(const boost::system::error_code& e,
92 ↪ boost::asio::steady_timer* t,
93                               boost::asio::io_context::strand* strand,
94                               ↪ MemoriaCompartilhada* shm)
95 {
96     if (e) return;
97     if (shm->c_encerrar) {
98         log_message("COMANDO", "Encerrando comando de navegação.");
99         return;
100     }
101
102     // Traduz comandos em estados | manual/automático
103     if (shm->c_automatigo) {
104         shm->e_automatigo = true;
105         shm->c_automatigo = false;
106         log_message("COMANDO", "Modo AUTOMÁTICO ativado");
107     }
108     if (shm->c_man) {
109         shm->e_automatigo = false;
110         shm->c_man = false;
111         shm->o_aceleracao = 0; // cancela qualquer desaceleração de inspeção
112         log_message("COMANDO", "Modo MANUAL ativado");
113     }
114
115     reagendar_tarefa(t, PERIODO_COMANDO, "COMANDO");
116     t->async_wait(boost::asio::bind_executor(*strand,
117 ↪ std::bind(comando_navegacao, std::placeholders::_1, t, strand, shm)));
118 }
119
120 void controle_navegacao(const boost::system::error_code& e,
121 ↪ boost::asio::steady_timer* t,
122                               boost::asio::io_context::strand* strand,
123                               ↪ MemoriaCompartilhada* shm, VarCondSinc &sinc)
124 {
125     if (e) return;
126     if (shm->c_encerrar) return;
127
128     bool processou_algo = false;

```

```

127 {
128     std::lock_guard<std::mutex> lock(sinc.mtx_navegacao);
129     while (sinc.disp_naveg_c > 0) {
130         sinc.LER_IDX_NAVEG_C = (sinc.LER_IDX_NAVEG_C + 1) %
            ↪ ELEMENTOS_BUFFERS;
131         sinc.disp_naveg_c--;
132         processou_algo = true;
133     }
134 }
135
136 if (processou_algo) {
137     executado[0]++;
138 } else {
139     miss[0]++;
140 }
141
142 // Controlador PID de velocidade
143 // Computa P, I e D a partir do encoder. Em um sistema embarcado real a
144 // saída acionaria o driver de motor; aqui a simulação Python já gerencia
            ↪ a
145 // física da velocidade, então a saída é calculada mas não sobrescreve
146 // o_aceleracao (que é exclusivo de reconstrucao_teto para inspeção).
147 if (shm->e_automatico && s_encoder_ativo) {
148     static float integral = 0.0f;
149     static float erro_ant = 0.0f;
150     static bool insp_ant = false;
151
152     constexpr float DT = PERIODO_CONTROLE / 1000.0f;
153     constexpr float Kp = 5.0f, Ki = 1.0f, Kd = 0.2f;
154
155     // Zera integrador ao sair de inspeção para evitar windup residual
156     if (insp_ant && !shm->e_inspecao) { integral = 0.0f; erro_ant = 0.0f; }
157     insp_ant = shm->e_inspecao;
158
159     float sp_ms = shm->j_sp_velocidade * 0.04f; // sp=50 → 2.0 m/s
160     float erro = sp_ms - s_vel_encoder_ms;
161
162     integral = std::clamp(integral + erro * DT, -20.0f, 20.0f);
163     float derivada = (erro - erro_ant) / DT;
164     erro_ant = erro;
165
166     [[maybe_unused]] float saida_pid =
167         std::clamp(Kp * erro + Ki * integral + Kd * derivada, -100.0f,
            ↪ 100.0f);
168 }
169
170
171 reagendar_tarefa(t, PERIODO_CONTROLE, "CONTROLE");
172 t->async_wait(boost::asio::bind_executor(*strand,
173     std::bind(controle_navegacao, std::placeholders::_1, t, strand, shm,
            ↪ std::ref(sinc))));

```

```

174 }
175
176
177 void distancia_percorrida(const boost::system::error_code& e,
    ↪ boost::asio::steady_timer* t,
178                             boost::asio::io_context::strand* strand,
    ↪ MemoriaCompartilhada* shm, VarCondSinc &sinc)
179 {
180     if (e) return;
181     if (shm->c_encerrar) return;
182
183     // Detecta transição do encoder e acumula distância em metros
184     static bool ultimo_encoder = false;
185     bool encoder_atual = shm->i_encoder;
186     auto agora_enc = std::chrono::steady_clock::now();
187
188     if (encoder_atual != ultimo_encoder) {
189         // Nova transição: calcula velocidade = 1 metro / tempo desde última
    ↪ transição
190         if (s_encoder_ativo) {
191             float dt = std::chrono::duration<float>(agora_enc -
    ↪ s_t_ultimo_encoder).count();
192             if (dt > 0.01f)
193                 s_vel_encoder_ms = 1.0f / dt;
194         }
195         s_t_ultimo_encoder = agora_enc;
196         s_encoder_ativo = true;
197         // posicao_x é um odômetro de distância percorrida, não de
    ↪ deslocamento:
198         // sempre incrementa independentemente da direção. Em modo manual
    ↪ remoto,
199         // recuar também aumenta posicao_x pois o encoder não carrega sinal de
    ↪ direção.
200         shm->posicao_x += 1.0f;
201         ultimo_encoder = encoder_atual;
202     } else if (s_encoder_ativo) {
203         // Entre transições: velocidade estimada decai naturalmente com o
    ↪ tempo
204         float dt = std::chrono::duration<float>(agora_enc -
    ↪ s_t_ultimo_encoder).count();
205         if (dt > 0.01f)
206             s_vel_encoder_ms = 1.0f / dt;
207     }
208
209     {
210         std::lock_guard<std::mutex> lock(sinc.mtx_navegacao);
211
212         if (sinc.disp_naveg_c >= ELEMENTOS_BUFFERS) {
213             sinc.LER_IDX_NAVEG_C = (sinc.LER_IDX_NAVEG_C + 1) %
    ↪ ELEMENTOS_BUFFERS;
214             sinc.disp_naveg_c--;

```

```

215     }
216     if (sinc.disp_naveg_r >= ELEMENTOS_BUFFERS) {
217         sinc.LER_IDX_NAVEG_R = (sinc.LER_IDX_NAVEG_R + 1) %
            ↳ ELEMENTOS_BUFFERS;
218         sinc.disp_naveg_r--;
219     }
220
221     // Escreve a posição atual para controle e reconstrução
222     sinc.BUFFER_NAVEGACAO[sinc.ESC_IDX_NAVEG] = shm->posicao_x;
223     sinc.ESC_IDX_NAVEG = (sinc.ESC_IDX_NAVEG + 1) % ELEMENTOS_BUFFERS;
224
225     sinc.disp_naveg_c++;
226     sinc.disp_naveg_r++;
227 }
228
229 executado[1]++;
230
231 reagendar_tarefa(t, PERIODO_DISTANCIA, "DISTANCIA");
232 t->async_wait(boost::asio::bind_executor(*strand,
233     std::bind(distancia_percorrida, std::placeholders::_1, t, strand, shm,
            ↳ std::ref(sinc))));
234 }
235
236
237 void reconstrucao_teto(const boost::system::error_code& e,
            ↳ boost::asio::steady_timer* t,
238     boost::asio::io_context::strand* strand,
            ↳ MemoriaCompartilhada* shm, VarCondSinc &sinc)
239 {
240     if (e) return;
241     if (shm->c_encerrar) return;
242
243     // Consome atualizações de posição do buffer de navegação
244     bool houve_atualizacao = false;
245     {
246         std::lock_guard<std::mutex> lock_nav(sinc.mtx_navegacao);
247         while (sinc.disp_naveg_r > 0) {
248             sinc.LER_IDX_NAVEG_R = (sinc.LER_IDX_NAVEG_R + 1) %
                ↳ ELEMENTOS_BUFFERS;
249             sinc.disp_naveg_r--;
250             houve_atualizacao = true;
251         }
252     }
253
254     if (houve_atualizacao) {
255         // Filtro de média móvel (janela de 2 amostras) sobre o LIDAR
256         static float janela[2] = {};
257         static int idx_janela = 0;
258         static bool janela_cheia = false;
259
260         janela[idx_janela % 2] = static_cast<float>(shm->i_lidar);

```

```

261     idx_janela++;
262     if (idx_janela >= 2) janela_cheia = true;
263
264     int n = janela_cheia ? 2 : idx_janela;
265     float soma = 0.0f;
266     for (int i = 0; i < n; ++i) soma += janela[i];
267     float media = soma / n;
268
269     // Escreve valor filtrado no buffer de nível
270     {
271         std::lock_guard<std::mutex> lock_nivel(sinc.mtx_nivel);
272
273         // Estabelece baseline a partir das primeiras 15 leituras (antes de
274         ↪ qualquer anomalia)
275         // Depois compara cada leitura contra esse baseline para detectar
276         ↪ variação severa
277         static float baseline = 0.0f;
278         static float baseline_soma = 0.0f;
279         static int baseline_n = 0;
280         const int BASELINE_AMOSTRAS = 15;
281
282         if (baseline_n < BASELINE_AMOSTRAS) {
283             // Aguarda janela cheia e média estabilizada (> 100) para
284             ↪ excluir zeros de startup
285             // e valores de transição como (0+0+0+0+145)/5 = 29 que
286             ↪ corrompem o baseline
287             if (janela_cheia && media > 100.0f) {
288                 baseline_soma += media;
289                 baseline_n++;
290                 if (baseline_n == BASELINE_AMOSTRAS) {
291                     baseline = baseline_soma / BASELINE_AMOSTRAS;
292                     log_message("RECONSTRUCAO", "Baseline estabelecido: " +
293                     ↪ std::to_string(baseline));
294                 }
295             }
296         } else {
297             float variacao = std::abs(media - baseline);
298             if (variacao > shm->variacao_severa) {
299                 if (!shm->e_inspecao) { // dispara apenas uma vez por
300                 ↪ anomalia
301                     shm->e_inspecao = true;
302                     shm->o_liga_camera = true;
303                     if (shm->e_automatico) // desacelera apenas em modo
304                     ↪ automático
305                         shm->o_aceleracao = -30;
306                     kill(getpid(), SIGUSR1);
307                     log_message("RECONSTRUCAO", "Variacao severa detectada:
308                     ↪ " + std::to_string(variacao));
309                 }
310             } else if (shm->e_inspecao) { // anomalia superada: restaura
311             ↪ estado normal

```

```

303         shm->e_inspecao = false;
304         shm->o_liga_camera = false;
305         shm->o_aceleracao = 0;
306         log_message("RECONSTRUCAO", "Anomalia superada, retomando
        ↳ velocidade normal");
307     }
308 }
309
310     if (sinc.disp_nivel >= ELEMENTOS_BUFFERS) {
311         sinc.LER_IDX_NIVEL = (sinc.LER_IDX_NIVEL + 1) %
        ↳ ELEMENTOS_BUFFERS;
312         sinc.disp_nivel--;
313     }
314     sinc.BUFFER_NIVEL[sinc.ESC_IDX_NIVEL] = media;
315     sinc.ESC_IDX_NIVEL = (sinc.ESC_IDX_NIVEL + 1) % ELEMENTOS_BUFFERS;
316     sinc.disp_nivel++;
317
318     // Sinaliza para o publisher MQTT que há novo ponto de perfil
319     shm->perfil_y = media;
320     shm->perfil_novo = true;
321 }
322
323     executado[2]++;
324 } else {
325     miss[2]++;
326 }
327
328 reagendar_tarefa(t, PERIODO_RECONSTRUCAO, "RECONSTRUCAO");
329 t->async_wait(boost::asio::bind_executor(*strand,
330     std::bind(reconstrucao_teto, std::placeholders::_1, t, strand, shm,
        ↳ std::ref(sinc))));
331 }
332
333
334 void coletor_dados(const boost::system::error_code& e,
    ↳ boost::asio::steady_timer* t,
335     boost::asio::io_context::strand* strand,
        ↳ MemoriaCompartilhada* shm, VarCondSinc &sinc)
336 {
337     static FILE* arquivo_coletor = nullptr;
338     static bool arquivo_aberto = false;
339     static std::size_t contador_coletor = 0;
340     static const auto inicio_coletor = std::chrono::steady_clock::now();
341
342     // Variáveis para cálculo online de confiança
343     // Mais medições numa mesma zona de posição ↳ maior confiança
344     static float zona_x_anterior = -999.0f;
345     static int contagem_zona = 0;
346
347     if (e || shm->c_encerrar) {
348         if (arquivo_aberto) {

```

```

349         std::fflush(arquivo_coletor);
350         std::fclose(arquivo_coletor);
351         arquivo_aberto = false;
352         arquivo_coletor = nullptr;
353         log_message("COLETOR", "Arquivo de coleta fechado.");
354     }
355     return;
356 }
357
358 if (!arquivo_aberto) {
359     arquivo_coletor = std::fopen("dados_coletados.csv", "a+");
360     arquivo_aberto = true;
361     if (arquivo_coletor == nullptr) {
362         log_message("COLETOR", "Erro ao abrir arquivo de coleta!");
363     } else {
364         log_message("COLETOR", "Arquivo de coleta aberto para escrita.");
365         std::fseek(arquivo_coletor, 0, SEEK_END);
366         if (std::ftell(arquivo_coletor) == 0) {
367             std::fprintf(arquivo_coletor,
368                 "timestamp_ms;posicao_x;teto_filtrado_y;lidar_raw;"
369
370                 ↪ "encoder;aceleracao;velocidade;e_automatico;e_inspecao;confianca\n")
371             std::fflush(arquivo_coletor);
372         }
373     }
374 }
375 std::vector<float> dados_coletados;
376 {
377     std::lock_guard<std::mutex> lock(sinc.mtx_nivel);
378     while (sinc.disp_nivel > 0) {
379         dados_coletados.push_back(sinc.BUFFER_NIVEL[sinc.LER_IDX_NIVEL]);
380         sinc.LER_IDX_NIVEL = (sinc.LER_IDX_NIVEL + 1) % ELEMENTOS_BUFFERS;
381         sinc.disp_nivel--;
382     }
383 }
384
385 if (!dados_coletados.empty()) {
386     executado[3]++;
387
388     // Cálculo online de confiança: mais medições na mesma zona + maior
389     ↪ confiança
390     if (std::abs(shm->posicao_x - zona_x_anterior) < 1.0f) {
391         contagem_zona = std::min(contagem_zona + 1, 10);
392     } else {
393         zona_x_anterior = shm->posicao_x;
394         contagem_zona = 1;
395     }
396     float confianca = contagem_zona / 10.0f;
397     shm->perfil_confianca = confianca; // disponibiliza para o publisher
398     ↪ MQTT

```



```

397
398 // Média de todas as leituras acumuladas no buffer / evita duplicatas e
    ↳ não perde passos
399 float dado = 0.0f;
400 for (float v : dados_coletados) dado += v;
401 dado /= static_cast<float>(dados_coletados.size());
402 const auto agora = std::chrono::steady_clock::now();
403 const long long timestamp_ms =
404     std::chrono::duration_cast<std::chrono::milliseconds>(agora -
    ↳ inicio_coletor).count();
405
406 if (arquivo_coletor) {
407     const int velocidade_efetiva = shm->j_sp_velocidade +
    ↳ shm->o_aceleracao;
408     std::fprintf(arquivo_coletor,
    ↳ "%lld;%.2f;%.2f;%d;%d;%d;%d;%d;%d;%.2f\n",
409         timestamp_ms,
410         shm->posicao_x,
411         dado,
412         static_cast<int>(shm->i_lidar),
413         static_cast<int>(shm->i_encoder),
414         static_cast<int>(shm->o_aceleracao),
415         velocidade_efetiva,
416         static_cast<int>(shm->e_automatico),
417         static_cast<int>(shm->e_inspecao),
418         confianca);
419
420     if (++contador_coletor >= 10) {
421         contador_coletor = 0;
422         std::fflush(arquivo_coletor);
423     }
424 }
425 } else {
426     miss[3]++;
427 }
428
429 if ((executado[3] + miss[3]) % 100 == 0) {
430     std::cout << "[STATUS] Miss: ";
431     for (auto i : miss) std::cout << i << " ";
432     std::cout << "| Exec: ";
433     for (auto i : executado) std::cout << i << " ";
434     std::cout << "\n";
435 }
436
437 reagendar_tarefa(t, PERIODO_COLETOR, "COLETOR");
438 t->async_wait(boost::asio::bind_executor(*strand,
439     std::bind(coletor_dados, std::placeholders::_1, t, strand, shm,
    ↳ std::ref(sinc))));
440 }
441
442

```

```

443 void inspecao_camera(const boost::system::error_code& e,
    ↪ boost::asio::steady_timer* t,
444         boost::asio::io_context::strand* strand, std::mutex& mtx,
    ↪ MemoriaCompartilhada* shm) {
445     if (e) return;
446     if (shm->c_encerrar) return;
447
448     {
449         std::lock_guard<std::mutex> lock(mtx);
450         if (eventos_camera > 0) {
451             eventos_camera--;
452
453             log_message("CAMERA", "Inspeção iniciada | processamento de imagem
    ↪ em curso");
454
455             // Simula carga de CPU real: análise de imagem por visão
    ↪ computacional embarcada
456             volatile float acumulador = 0.0f;
457             for (int i = 1; i <= 800000; ++i) {
458                 acumulador += std::sqrt(static_cast<float>(i)) *
459                     std::sin(static_cast<float>(i) * 0.0001f);
460             }
461             (void)acumulador; // impede otimização do compilador
462
463             log_message("CAMERA", "Inspeção finalizada");
464         }
465     }
466
467     reagendar_tarefa(t, PERIODO_CAMERA, "CAMERA");
468     t->async_wait(boost::asio::bind_executor(*strand,
469         std::bind(inspecao_camera, std::placeholders::_1, t, strand,
    ↪ std::ref(mtx), shm)));
470 }

```

8.5 sincronizacao.hpp

```

1  #ifndef SINCRONIZACAO_H
2  #define SINCRONIZACAO_H
3
4  #include "includes.hpp"
5  #include "shared_memory.hpp"
6
7  #define ELEMENTOS_BUFFERS 10
8
9  #define PERIODO_COMANDO 500
10 #define PERIODO_CONTROLE 80
11 #define PERIODO_DISTANCIA 20
12 #define PERIODO_CAMERA 80
13 #define PERIODO_COLETOR 100
14 #define PERIODO_RECONSTRUCAO 80
15

```

```

16 struct VarCondSinc {
17     // BUFFER 1: NAVEGAÇÃO
18     std::vector<float> BUFFER_NAVEGACAO =
19         ↪ std::vector<float>(ELEMENTOS_BUFFERS);
20     int ESC_IDX_NAVEG = 0;    // Onde o produtor escreve
21     int LER_IDX_NAVEG_C = 0;  // Onde o Controle lê
22     int LER_IDX_NAVEG_R = 0;  // Onde a Reconstrução lê
23
24     int disp_naveg_c = 0;     // Quantidade de dados pendentes para o
25         ↪ Controle
26     int disp_naveg_r = 0;     // Quantidade de dados pendentes para a
27         ↪ Reconstrução
28     std::mutex mtx_navegacao; // Proteção exclusiva deste buffer
29
30     // === BUFFER 2: NÍVEL ===
31     std::vector<float> BUFFER_NIVEL = std::vector<float>(ELEMENTOS_BUFFERS);
32     int ESC_IDX_NIVEL = 0;    // Onde a Reconstrução escreve
33     int LER_IDX_NIVEL = 0;    // Onde o Coletor lê
34
35     int disp_nivel = 0;       // Quantidade de dados pendentes para o Coletor
36     std::mutex mtx_nivel;     // Proteção exclusiva deste buffer
37
38     // === CÂMERA ===
39     std::mutex mutex_camera;
40 };
41
42 typedef boost::asio::steady_timer tempo_tarefa;
43 typedef boost::asio::chrono::milliseconds milisegundos;
44
45 void log_message (const std::string& thread, const std::string& mensagem);
46 float numero_aleatorio_debugg();
47
48 void handler_signal (const boost::system::error_code& error, int signal_number,
49     ↪
50
51         boost::asio::steady_timer* t,
52         ↪ boost::asio::io_context::strand* strand_camera,
53         std::mutex& mtx, MemoriaCompartilhada* shm,
54         ↪ boost::asio::signal_set* sinais);
55
56 /**
57  * @brief Tarefa que recebe comandos do sistema de operação remoto e traduz os
58  * ↪ comandos em setpoint de velocidade
59  * e liga/desliga para o controle de navegação. O comando de navegação que
60  * ↪ deve implementar a lógica de manual/
61  * automático. Exerce a função de ESCRITOR sobre o BUFFER_NAVEGACAO
62  *
63  */
64 void comando_navegacao(const boost::system::error_code& /*e*/,
65     boost::asio::steady_timer* t,
66     boost::asio::io_context::strand* strand,
67     MemoriaCompartilhada* shm);

```

```

59
60 /**
61  * @brief implementação de um controlador PID responsável pelo acionamento dos
62  * ↳ motores (controle de velocidade)
63  *
64  * a partir de um setpoint de velocidade recebido pelo Comando de Navegação.
65  * ↳ Exerce a função de LEITOR do BUFFER _NAVEGACAO
66  *
67  * @param mtx mutex utilizado
68  * @param BUFFER historico de posições
69  */
70 void controle_navegacao(const boost::system::error_code& e,
71                          boost::asio::steady_timer* t,
72                          boost::asio::io_context::strand* strand,
73                          MemoriaCompartilhada* shm, VarCondSinc &sinc);
74
75 /**
76  * @brief Registra a distância percorrida fornecida pelo encoder. Possui a
77  * ↳ função de ESCRITOR sobre o BUFFER_NAVEGACAO.
78  *
79  * @param mtx mutex utilizado na variavel compartilhada
80  * @param BUFFER historico de posições
81  */
82 void distancia_percorrida(const boost::system::error_code& e,
83                           boost::asio::steady_timer* t,
84                           boost::asio::io_context::strand* strand,
85                           MemoriaCompartilhada* shm, VarCondSinc &sinc);
86
87 /**
88  * @brief Recebe os valores do lidar para reconstruir o teto do túnel. Atua
89  * ↳ como
90  * ESCRITOR do BUFFER_NIVEL
91  *
92  * @param mtx mutex para buffer compartilhado
93  * @param BUFFER vetor de dados do nível da distancia do teto
94  */
95 void reconstrucao_teto(const boost::system::error_code& e,
96                        boost::asio::steady_timer* t,
97                        boost::asio::io_context::strand* strand,
98                        MemoriaCompartilhada* shm, VarCondSinc &sinc);
99
100 /**
101  * @brief Registra os dados coletados pelo lidar num Banco de Dados. Atua como
102  * ↳ LEITOR do BUFFER_NIVEL.
103  *
104  * @param mtx mutex para sincronizar o buffer compartilhado
105  * @param BUFFER buffer de dados coletados pelo lidar
106  */
107 void coletor_dados(const boost::system::error_code& e,
108                   boost::asio::steady_timer* t,
109                   boost::asio::io_context::strand* strand,
110                   MemoriaCompartilhada* shm, VarCondSinc &sinc);

```

```

105 void inspecao_camera(const boost::system::error_code& e,
106                     boost::asio::steady_timer* t,
107                     boost::asio::io_context::strand* strand,
108                     std::mutex& mtx, MemoriaCompartilhada* shm);
109
110 void operacao_remota(std::mutex &mtx, std::vector <float> &BUFFER);
111
112 #endif

```

8.6 shared_memory.hpp

```

1  #ifndef SHARED_MEMORY_HPP
2  #define SHARED_MEMORY_HPP
3
4  #include "include/includes.hpp"
5  #include <cstdint>
6
7  constexpr const char* SHM_FILE = "/tmp/memoria_compartilhada.bin";
8
9  struct MemoriaCompartilhada {
10
11     //sensores e atuadores disponíveis no carrinho:
12     bool i_encoder; //Variável que simula a entrada de um encoder, que troca de
13     ↪ estado a cada metro percorrido pelo robô
14     int32_t i_lidar; //Resposta do sensor LIDAR do veículo exibindo a distância
15     ↪ no eixo y, com relação à altura do robô
16     bool o_liga_camera; //Comando para ligar a câmera e realizar fotos da falha
17     ↪ detectada na superfície
18     int32_t o_aceleracao; //Determina a aceleração do veículo em percentual
19     ↪ (-100 a 100%)
20
21     //estados e comandos:
22     bool e_inspecao; //Estado que indica falha detectada na superfície e o robô
23     ↪ está tirando fotos e navegando com velocidade limitada (1:falha, 0: sem
24     ↪ falha)
25     bool e_automatico; //Estado que identifica o modo de operação do robô (0:
26     ↪ manual, 1: automático)
27     bool c_automatico; //Comando para passar o robô para o modo automático
28     ↪ (true). O reset desse comando
29     bool c_man; //Comando para passar o robô para o modo manual (true).
30     int32_t j_sp_velocidade; //Setpoint de velocidade do robô para o
31     ↪ controlador de velocidade.
32
33     bool c_encerrar; //variável que sinaliza ao programa que a interface foi
34     ↪ fechada
35
36     // Parâmetros configuráveis via MQTT
37     float variacao_severa; // limiar de variação do teto que dispara inspeção
38     ↪ (configurável remotamente)
39
40     // Dados de perfil do teto para publicação MQTT

```

```

30     float posicao_x;           // posição acumulada do robô em metros (atualizada
    ↪     pelo encoder)
31     float perfil_y;           // última leitura filtrada do teto (média móvel)
32     float perfil_confianca;    // nível de confiança da medição (0.0 a 1.0)
33     bool  perfil_novo;         // sinaliza ao publisher que há novo ponto de
    ↪     perfil disponível
34 };
35
36 #endif

```

8.7 operacao_remota.cpp

```

1  #include "../include/includes.hpp"
2
3  void operacao_remota(std::mutex& mtx, std::vector<float>& BUFFER){
4      std::unique_lock<std::mutex> lock(mtx); // Protocolo de entrada
5      lock.unlock(); // Protocolo de saída
6  }

```

8.8 Makefile

```

1  TARGET = programa
2
3  CXX = g++
4
5  CXXFLAGS = -std=c++17 -Wall -Wextra -I. -Iinclude -I/opt/homebrew/include
    ↪ -I/usr/local/include -I/opt/homebrew/opt/boost/include -pthread
6
7  LDFLAGS = -L/opt/homebrew/lib -L/usr/local/lib \
8  -lpaho-mqttpp3 -lpaho-mqtt3c -lpthread \
9  -Wl,-rpath,/opt/homebrew/lib \
10 -Wl,-rpath,/usr/local/lib
11
12 SRCS = main.cpp src/sincronizacao.cpp src/simulacao.cpp src/mqtt_client.cpp
    ↪ src/mqtt_publisher.cpp src/mqtt_subscriber.cpp src/mqtt_manager.cpp
13
14 OBJS = $(SRCS:.cpp=.o)
15
16 all: $(TARGET)
17
18 $(TARGET): $(OBJS)
19     $(CXX) $(CXXFLAGS) $(OBJS) $(LDFLAGS) -o $(TARGET)
20
21 %.o: %.cpp
22     $(CXX) $(CXXFLAGS) -c $< -o $@
23
24 run: $(TARGET)
25     ./$$(TARGET)
26
27 test: src/test_sistema.cpp
28     $(CXX) $(CXXFLAGS) src/test_sistema.cpp -o test_sistema

```

```
29         ./test_sistema
30         python3 src/test_sistema.py
31
32     clean:
33         rm -f $(OBJS) $(TARGET) test_sistema
34
35     rebuild: clean all
```